```
"""JobsComing backend application package."""
"""API 路由包。"""
"""平台帳號加密庫（FR-1d / 主程序同步 GAP-2）。

- 寫入：為用戶生成/取回 DEK → 信封加密 → 落庫（密文）
- 讀取：僅返回掩碼後的帳號標識，永不下發密文/明文
- 解密密文只發生在採集/投遞 Worker（P3/P4），通過 get_dek_for_user 取用
"""
from __future__ import annotations

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, Request
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.audit import log_audit
from app.core.auth import get_current_user
from app.core.security import (
    decrypt_field,
    encrypt_field,
    encrypt_json,
    get_dek_for_user,
)
from app.db import get_db
from app.schemas.accounts import PLATFORMS, AccountCreate, AccountUpdate

router = APIRouter(prefix="/accounts", tags=["accounts"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]


def _mask(identifier: str) -> str:
    if len(identifier) <= 4:
        return "****"
    return identifier[:2] + "****" + identifier[-2:]


@router.get("")
def list_accounts(user: CurrentUser, db: DB) -> list[dict]:
    rows = db.execute(
        text(
            "SELECT id, platform, label, login_identifier_enc, status, last_login_at, last_error
"
            "FROM platform_accounts WHERE user_id = :uid ORDER BY created_at"
        ),
        {"uid": user["id"]},
    ).mappings().all()
    dek = get_dek_for_user(db, user["id"])
    result = []
```

```
    for r in rows:
        result.append(
            {
                "id": str(r["id"]),
                "platform": r["platform"],
                "label": r["label"],
                "login_masked": _mask(decrypt_field(dek, r["login_identifier_enc"])),
                "status": r["status"],
                "last_login_at": r["last_login_at"].isoformat() if r["last_login_at"] else None,
                "last_error": r["last_error"],
            }
        )
    return result


@router.post("", status_code=201)
def create_account(user: CurrentUser, db: DB, payload: AccountCreate, request: Request) -> dict:
    if payload.platform not in PLATFORMS:
        raise HTTPException(422, f"platform 必須是 {PLATFORMS} 之一")
    dek = get_dek_for_user(db, user["id"])
    row_id = db.execute(
        text(
            """
            INSERT INTO platform_accounts
              (user_id, platform, label, login_identifier_enc, password_enc, meta_enc)
            VALUES (:uid, :platform, :label, :ident, :pwd, :meta)
            RETURNING id
            """
        ),
        {
            "uid": user["id"],
            "platform": payload.platform,
            "label": payload.label,
            "ident": encrypt_field(dek, payload.login_identifier),
            "pwd": encrypt_field(dek, payload.password),
            "meta": encrypt_json(dek, payload.meta) if payload.meta is not None else None,
        },
    ).mappings().one()
    db.commit()
    log_audit(db, user["id"], "account.create", "platform_accounts", str(row_id["id"]),
              {"platform": payload.platform}, request.client.host if request.client else None)
    return {"id": str(row_id["id"])}


@router.patch("/{account_id}")
def update_account(
    user: CurrentUser, db: DB, account_id: str, payload: AccountUpdate, request: Request
) -> dict:
    exists = db.execute(
        text("SELECT 1 FROM platform_accounts WHERE id = :aid AND user_id = :uid"),
```

```
        {"aid": account_id, "uid": user["id"]},
    ).first()
    if not exists:
        raise HTTPException(404, "帳號不存在")
    dek = get_dek_for_user(db, user["id"])
    fields = payload.model_dump(exclude_none=True)
    sets, params = [], {"aid": account_id, "uid": user["id"]}
    if "label" in fields:
        sets.append("label = :label")
        params["label"] = fields["label"]
    if "login_identifier" in fields:
        sets.append("login_identifier_enc = :ident")
        params["ident"] = encrypt_field(dek, fields["login_identifier"])
    if "password" in fields:
        sets.append("password_enc = :pwd")
        params["pwd"] = encrypt_field(dek, fields["password"])
    if "meta" in fields:
        sets.append("meta_enc = :meta")
        params["meta"] = encrypt_json(dek, fields["meta"]) if fields["meta"] is not None else
None
    if sets:
        sets.append("status = 'pending_verify'")  # 憑據變更後需重新驗證登錄態
        db.execute(
            text(
                f"UPDATE platform_accounts SET {', '.join(sets)}, updated_at = now() "
                "WHERE id = :aid AND user_id = :uid"
            ),
            params,
        )
        db.commit()
        log_audit(db, user["id"], "account.update", "platform_accounts", account_id,
                  {}, request.client.host if request.client else None)
    return {"ok": True}


@router.delete("/{account_id}", status_code=204)
def delete_account(user: CurrentUser, db: DB, account_id: str, request: Request) -> None:
    row = db.execute(
        text("DELETE FROM platform_accounts WHERE id = :aid AND user_id = :uid RETURNING id"),
        {"aid": account_id, "uid": user["id"]},
    ).first()
    if not row:
        raise HTTPException(404, "帳號不存在")
    db.commit()
    log_audit(db, user["id"], "account.delete", "platform_accounts", account_id,
              {}, request.client.host if request.client else None)
"""管理後台端點（GAP-10）：用戶審計、適配器健康、投遞成功率、LLM 用量、審計日誌。"""
from __future__ import annotations

from typing import Annotated
```

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.auth import get_current_user
from app.core.llm import usage as llm_usage
from app.db import get_db

router = APIRouter(prefix="/admin", tags=["admin"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]


def require_admin(user: CurrentUser) -> dict:
    if user.get("role") != "admin":
        raise HTTPException(403, "需要管理員權限")
    return user


Admin = Annotated[dict, Depends(require_admin)]


@router.get("/stats")
def stats(admin: Admin, db: DB) -> dict:
    one = lambda sql, **kw: db.execute(text(sql), kw).mappings().one()  # noqa: E731
    users = one("SELECT COUNT(*) AS n FROM profiles WHERE deleted_at IS NULL")["n"]
    active_postings = one("SELECT COUNT(*) AS n FROM job_postings WHERE is_active=true")["n"]
    matches_24h = one(
        "SELECT COUNT(*) AS n FROM job_matches WHERE created_at >= now() - interval '24 hours'"
    )["n"]
    apps = one(
        """
        SELECT COUNT(*) FILTER (WHERE submitted_at >= now() - interval '24 hours') AS
submitted_24h,
               COUNT(*) AS total FROM applications
        """
    )
    open_tasks = one("SELECT COUNT(*) AS n FROM manual_tasks WHERE status='open'")["n"]
    accounts_issues = one(
        "SELECT COUNT(*) AS n FROM platform_accounts WHERE status IN
('expired','error','locked')"
    )["n"]
    return {
        "users": users,
        "active_postings": active_postings,
        "matches_24h": matches_24h,
        "applications_total": apps["total"],
        "applications_submitted_24h": apps["submitted_24h"],
        "open_manual_tasks": open_tasks,
        "platform_accounts_issue": accounts_issues,
        "llm_usage": llm_usage(),
```

```
        "collected_today": one(
            "SELECT COUNT(*) AS n FROM job_postings WHERE first_seen_at::date = now()::date"
        )["n"],
    }


@router.get("/users")
def list_users(admin: Admin, db: DB) -> list[dict]:
    rows = db.execute(
        text(
            """
            SELECT p.id, p.email, p.phone, p.role, p.created_at, p.deleted_at,
                   (SELECT COUNT(*) FROM applications a WHERE a.user_id=p.id) AS apps,
                   (SELECT COUNT(*) FROM job_matches m WHERE m.user_id=p.id) AS matches
            FROM profiles p ORDER BY p.created_at DESC LIMIT 500
            """
        )
    ).mappings().all()
    return [
        {
            "id": str(r["id"]), "email": r["email"], "phone": r["phone"], "role": r["role"],
            "created_at": r["created_at"].isoformat(),
            "disabled": r["deleted_at"] is not None,
            "applications": r["apps"], "matches": r["matches"],
        }
        for r in rows
    ]


@router.patch("/users/{user_id}")
def set_user(admin: Admin, db: DB, user_id: str, disabled: bool | None = None,
             role: str | None = None) -> dict:
    sets, params = [], {"id": user_id}
    if disabled is not None:
        sets.append("deleted_at = " + ("now()" if disabled else "NULL"))
    if role in ("user", "admin"):
        sets.append("role = :role")
        params["role"] = role
    if not sets:
        raise HTTPException(422, "無有效變更")
    row = db.execute(
        text(f"UPDATE profiles SET {', '.join(sets)} WHERE id=:id RETURNING id"),
        params,
    ).first()
    if not row:
        raise HTTPException(404, "用戶不存在")
    db.commit()
    return {"ok": True}
```

```
@router.get("/adapters")
def adapters(admin: Admin, db: DB) -> list[dict]:
    rows = db.execute(
        text("SELECT * FROM adapter_health ORDER BY adapter")
    ).mappings().all()
    return [
        {
            "adapter": r["adapter"], "last_run_at": r["last_run_at"].isoformat() if
r["last_run_at"] else None,
            "last_success_at": r["last_success_at"].isoformat() if r["last_success_at"] else
None,
            "success_count": r["success_count"], "fail_count": r["fail_count"],
            "last_error": r["last_error"], "avg_duration_ms": r["avg_duration_ms"],
        }
        for r in rows
    ]


@router.get("/audit")
def audit(admin: Admin, db: DB, limit: int = 100) -> list[dict]:
    rows = db.execute(
        text(
            """
            SELECT a.created_at, a.action, a.entity, a.entity_id, a.meta, p.email
            FROM audit_logs a LEFT JOIN profiles p ON p.id = a.actor_user_id
            ORDER BY a.created_at DESC LIMIT :lim
            """
        ),
        {"lim": min(limit, 500)},
    ).mappings().all()
    return [
        {
            "at": r["created_at"].isoformat(), "actor": r["email"], "action": r["action"],
            "entity": r["entity"], "entity_id": r["entity_id"], "meta": r["meta"],
        }
        for r in rows
    ]


@router.get("/collect-jobs")
def collect_jobs(admin: Admin, db: DB, limit: int = 50) -> list[dict]:
    rows = db.execute(
        text(
            "SELECT kind, status, payload, scheduled_at, started_at, finished_at, error "
            "FROM collect_jobs ORDER BY created_at DESC LIMIT :lim"
        ),
        {"lim": min(limit, 200)},
    ).mappings().all()
    return [
        {
            "kind": r["kind"], "status": r["status"], "payload": r["payload"],
            "started_at": r["started_at"].isoformat() if r["started_at"] else None,
```

```
            "finished_at": r["finished_at"].isoformat() if r["finished_at"] else None,
            "error": r["error"],
        }
        for r in rows
    ]
"""投遞記錄與觸發端點（FR-3b/3c）。"""
from __future__ import annotations

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.auth import get_current_user
from app.db import get_db
from app.services.apply_engine import (
    create_applications_from_matches,
    process_queue,
    run_application,
)

router = APIRouter(prefix="/applications", tags=["applications"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]


@router.get("")
def list_applications(
    user: CurrentUser, db: DB, status: str | None = None,
    limit: int = 20, offset: int = 0,
) -> dict:
    limit = min(max(limit, 1), 100)
    rows = db.execute(
        text(
            """
            SELECT a.id, a.status, a.status_reason, a.attempts_count, a.next_retry_at,
                   a.submitted_at, a.external_ref, a.created_at,
                   p.company_name, p.title, p.location, p.apply_url, p.source,
                   m.match_score, rv.id AS resume_version_id
            FROM applications a
            JOIN job_postings p ON p.id = a.posting_id
            LEFT JOIN job_matches m ON m.id = a.match_id
            LEFT JOIN resume_versions rv ON rv.id = a.resume_version_id
            WHERE a.user_id = :uid AND (CAST(:st AS text) IS NULL OR CAST(a.status AS text)
= :st)
            ORDER BY a.created_at DESC
            LIMIT :lim OFFSET :off
            """
        ),
```

```
        {"uid": user["id"], "st": status, "lim": limit, "off": offset},
    ).mappings().all()
    total = db.execute(
        text(
            """
            SELECT COUNT(*) AS n FROM applications a
            WHERE a.user_id = :uid AND (CAST(:st AS text) IS NULL OR CAST(a.status AS text)
= :st)
            """
        ),
        {"uid": user["id"], "st": status},
    ).mappings().one()["n"]
    return {
        "items": [
            {
                "id": str(r["id"]),
                "status": r["status"],
                "status_reason": r["status_reason"],
                "attempts_count": r["attempts_count"],
                "next_retry_at": r["next_retry_at"].isoformat() if r["next_retry_at"] else None,
                "submitted_at": r["submitted_at"].isoformat() if r["submitted_at"] else None,
                "external_ref": r["external_ref"],
                "created_at": r["created_at"].isoformat(),
                "company_name": r["company_name"],
                "title": r["title"],
                "location": r["location"],
                "apply_url": r["apply_url"],
                "source": r["source"],
                "match_score": r["match_score"],
                "resume_version_id": str(r["resume_version_id"]) if r["resume_version_id"] else
None,
            }
            for r in rows
        ],
        "total": total,
        "limit": limit,
        "offset": offset,
    }


@router.post("/run", status_code=202)
def run_now(user: CurrentUser, db: DB) -> dict:
    """手動觸發：按當前設置從匹配結果生成投遞隊列並立即處理。"""
    created = create_applications_from_matches(db, user["id"])
    processed = process_queue(db, user_id=user["id"])
    return {"created": created, "processed": processed}


@router.post("/{application_id}/retry", status_code=202)
def retry_application(user: CurrentUser, db: DB, application_id: str) -> dict:
    row = db.execute(
        text("SELECT status FROM applications WHERE id=:id AND user_id=:uid"),
```

```
        {"id": application_id, "uid": user["id"]},
    ).mappings().first()
    if not row:
        raise HTTPException(404, "投遞記錄不存在")
    if row["status"] in ("submitted", "in_progress"):
        return {"skipped": row["status"]}
    out = run_application(db, application_id, force=True)
    return out


@router.post("/{application_id}/abort", status_code=200)
def abort_application(user: CurrentUser, db: DB, application_id: str) -> dict:
    row = db.execute(
        text(
            "UPDATE applications SET status='aborted', status_reason='用戶手動取消',
updated_at=now() "
            "WHERE id=:id AND user_id=:uid AND status NOT IN ('submitted','aborted') RETURNING
id"
        ),
        {"id": application_id, "uid": user["id"]},
    ).first()
    if not row:
        raise HTTPException(404, "投遞記錄不存在或已結束")
    db.commit()
    return {"ok": True}
"""認證代理（關鍵架構）：前端不直連 supabase.co，全部 auth 請求經後端轉發。

原因：部分地區網絡無法直達 *.supabase.co（實測本網絡連接被重置）。
後端部署於美區（Render），可正常訪問 Supabase Auth API。
"""
from __future__ import annotations

import httpx
from fastapi import APIRouter, HTTPException, Request
from pydantic import BaseModel

from app.config import settings

router = APIRouter(prefix="/auth", tags=["auth"])

_SB = settings.supabase_url.rstrip("/")


def _anon_key() -> str:
    return settings.supabase_anon_key


def _anon_headers() -> dict:
    k = _anon_key()
    return {"apikey": k, "Authorization": f"Bearer {k}", "Content-Type": "application/json"}
```

```
class Credentials(BaseModel):
    email: str
    password: str


class MagicLinkRequest(BaseModel):
    email: str


class RefreshRequest(BaseModel):
    refresh_token: str


@router.post("/signup")
def signup(payload: Credentials) -> dict:
    try:
        resp = httpx.post(
            f"{_SB}/auth/v1/signup", headers=_anon_headers(),
            json={"email": payload.email, "password": payload.password}, timeout=20,
        )
    except httpx.HTTPError as exc:
        raise HTTPException(502, f"無法連接 Supabase Auth（服務端網絡問題）: {exc}") from exc
    if resp.status_code >= 400:
        raise HTTPException(resp.status_code, _err(resp))
    data = resp.json()
    if not data.get("session"):
        return {"status": "email_confirmation_required", "user": data.get("user")}
    return {"status": "ok", "session": data["session"], "user": data["user"]}


@router.post("/signin")
def signin(payload: Credentials) -> dict:
    try:
        resp = httpx.post(
            f"{_SB}/auth/v1/token?grant_type=password", headers=_anon_headers(),
            json={"email": payload.email, "password": payload.password}, timeout=20,
        )
    except httpx.HTTPError as exc:
        raise HTTPException(502, f"無法連接 Supabase Auth（服務端網絡問題）: {exc}") from exc
    if resp.status_code >= 400:
        raise HTTPException(resp.status_code, _err(resp))
    return resp.json()


@router.post("/magic-link")
def magic_link(payload: MagicLinkRequest) -> dict:
    try:
        resp = httpx.post(
            f"{_SB}/auth/v1/magiclink", headers=_anon_headers(),
            json={"email": payload.email, "emailRedirectTo": settings.frontend_url}, timeout=20,
```

```
        )
    except httpx.HTTPError as exc:
        raise HTTPException(502, f"無法連接 Supabase Auth（服務端網絡問題）: {exc}") from exc
    if resp.status_code >= 400:
        raise HTTPException(resp.status_code, _err(resp))
    return {"status": "sent"}


@router.post("/refresh")
def refresh(payload: RefreshRequest) -> dict:
    try:
        resp = httpx.post(
            f"{_SB}/auth/v1/token?grant_type=refresh_token", headers=_anon_headers(),
            json={"refresh_token": payload.refresh_token}, timeout=20,
        )
    except httpx.HTTPError as exc:
        raise HTTPException(502, f"無法連接 Supabase Auth（服務端網絡問題）: {exc}") from exc
    if resp.status_code >= 400:
        raise HTTPException(resp.status_code, _err(resp))
    return resp.json()


class AdminCreate(BaseModel):
    email: str
    password: str
    secret: str


@router.post("/admin-create")
def admin_create(payload: AdminCreate) -> dict:
    """維護端點：用 service_role 走 GoTrue 管理 API 正規創建用戶（含郵箱確認）。

  僅供管理員運維使用，secret 由 OPS_SECRET 環境變量保護。
    """
    if not settings.ops_secret or payload.secret != settings.ops_secret:
        raise HTTPException(403, "forbidden")
    try:
        resp = httpx.post(
            f"{_SB}/auth/v1/admin/users",
            headers={
                "apikey": settings.supabase_service_role_key,
                "Authorization": f"Bearer {settings.supabase_service_role_key}",
                "Content-Type": "application/json",
            },
            json={"email": payload.email, "password": payload.password, "email_confirm": True},
            timeout=25,
        )
    except httpx.HTTPError as exc:
        raise HTTPException(502, f"無法連接 Supabase Auth: {exc}") from exc
    try:
```

```
        body = resp.json()
    except Exception:  # noqa: BLE001
        body = resp.text[:300]
    return {"status": resp.status_code, "body": body}


def _err(resp: httpx.Response) -> str:
    try:
        return resp.json().get("msg") or resp.json().get("error_description") or resp.text[:300]
    except Exception:  # noqa: BLE001
        return resp.text[:300]


class AdminResetRequest(BaseModel):
    secret: str
    email: str
    new_password: str


@router.post("/admin-reset-password")
def admin_reset_password(payload: AdminResetRequest) -> dict:
    """維護端點：管理員重置用戶密碼（service_role 管理 API）。"""
    if not settings.ops_secret or payload.secret != settings.ops_secret:
        raise HTTPException(403, "forbidden")
    if len(payload.new_password) < 6:
        raise HTTPException(422, "密碼至少 6 位")
    headers = {
        "apikey": settings.supabase_service_role_key,
        "Authorization": f"Bearer {settings.supabase_service_role_key}",
        "Content-Type": "application/json",
    }
    try:
        # 找用戶 id（分頁列出）
        user_id = None
        page = 1
        while page <= 10:
            r = httpx.get(
                f"{_SB}/auth/v1/admin/users?per_page=200&page={page}",
                headers=headers, timeout=25,
            )
            if r.status_code != 200:
                return {"status": r.status_code, "body": r.text[:300]}
            users = r.json().get("users") or []
            hit = next((u for u in users if (u.get("email") or "").lower() ==
payload.email.lower()), None)
            if hit:
                user_id = hit["id"]
                break
            if not users or len(users) < 200:
                break
            page += 1
```

```
        if not user_id:
            raise HTTPException(404, "找不到該郵箱用戶")
        r = httpx.put(
            f"{_SB}/auth/v1/admin/users/{user_id}",
            headers=headers,
            json={"password": payload.new_password},
            timeout=25,
        )
        if r.status_code != 200:
            return {"status": r.status_code, "body": r.text[:300]}
        return {"ok": True, "note": "密碼已重置"}
    except httpx.HTTPError as exc:
        raise HTTPException(502, f"無法連接 Supabase Auth: {exc}") from exc

class ChangePasswordRequest(BaseModel):
    old_password: str
    new_password: str


@router.post("/change-password")
def change_password(user: CurrentUser, db: DB, payload: ChangePasswordRequest, request: Request)
-> dict:
    """修改密碼：先驗證舊密碼，再經 GoTrue 更新（配合前端 E2EE 密鑰遷移）。"""
    import httpx as _httpx

    if len(payload.new_password) < 6:
        raise HTTPException(422, "新密碼至少 6 位")
    email = user.get("email")
    # 驗證舊密碼
    try:
        r = _httpx.post(
            f"{settings.supabase_url}/auth/v1/token?grant_type=password",
            headers={"apikey": settings.supabase_anon_key},
            json={"email": email, "password": payload.old_password},
            timeout=20,
        )
    except Exception as exc:  # noqa: BLE001
        raise HTTPException(502, f"驗證服務不可用: {exc}") from exc
    if r.status_code != 200:
        raise HTTPException(401, "舊密碼不正確")
    # 用當前用戶 token 更新密碼
    token = request.headers.get("Authorization", "").removeprefix("Bearer ").strip()
    r = _httpx.put(
        f"{settings.supabase_url}/auth/v1/user",
        headers={"apikey": settings.supabase_anon_key, "Authorization": f"Bearer {token}"},
        json={"password": payload.new_password},
        timeout=20,
    )
    if r.status_code != 200:
        raise HTTPException(502, f"密碼更新失敗: {r.text[:200]}")
    return {"ok": True, "note": "密碼已更新；敏感字段密文已由前端用新密鑰重新加密"}
```

```
"""AI 求職管家對話端點（首頁 Chatbot）。

LLM 通過工具（function calling）直接操作用戶數據：
查詢畫像/匹配/投遞/待處理、觸發採集、修改設置、首次使用引導。
"""
from __future__ import annotations

import logging
import re
from typing import Annotated

from fastapi import APIRouter, Depends
from pydantic import BaseModel, Field
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.collectors.pipeline import collect_for_user
from app.core.auth import get_current_user
from app.core.jsonutil import j
from app.core.textutil import t2s as _t2s_global
from app.core.llm import chat_json, chat_with_tools
from app.db import SessionLocal, get_db

# 繁→簡 對照（成對排列，索引偶數為繁、奇數為簡）
_T2S_MAP = {
    "產": "产", "品": "品", "經": "经", "理": "理", "開": "开", "發": "发",
    "驗": "验", "數": "数", "據": "据", "應": "应", "屆": "届", "培": "培",
    "訓": "训", "員": "员", "務": "务", "職": "职", "證": "证", "聯": "联",
    "網": "网", "點": "点", "類": "类", "銀": "银", "資": "资", "訊": "讯",
    "質": "质", "動": "动", "範": "范", "組": "组", "織": "织", "級": "级",
    "實": "实", "習": "习", "業": "业", "專": "专", "門": "门", "間": "间",
    "時": "时", "機": "机", "構": "构", "規": "规", "畫": "画", "計": "计",
    "設": "设", "銷": "销", "場": "场", "營": "营", "錢": "钱", "術": "术",
    "歷": "历", "係": "系", "國": "国", "體": "体", "單": "单", "與": "与",
    "讀": "读", "種": "种", "長": "长", "風": "风", "控": "控", "優": "优",
    "權": "权", "廣": "广", "東": "东", "亞": "亚", "險": "险", "斷": "断",
    "檔": "档", "待": "待", "校": "校", "園": "园", "崗": "岗", "簡": "简",
    "選": "选", "擇": "择", "請": "请", "填": "填", "寫": "写", "郵": "邮",
    "箱": "箱", "畢": "毕", "學": "学", "戶": "户", "籍": "籍", "城": "城",
    "市": "市", "農": "农", "村": "村", "金": "金", "融": "融", "練": "练",
    "續": "续", "處": "处", "號": "号", "臺": "台", "灣": "湾", "滬": "沪",
    "深": "深", "圳": "圳", "京": "京", "穗": "穗", "併": "并", "購": "购",
    "債": "债", "財": "财", "部": "部", "門": "门", "歐": "欧", "亞": "亚",
    "會": "会", "計": "计", "師": "师", "簽": "签", "證": "证", "顯": "显",
    "離": "离", "復": "复", "資": "资", "訊": "讯", "億": "亿", "萬": "万",
    "說明": "说明", "適用": "适用", "關係": "关系", "責任": "责任",
}
```

```
logger = logging.getLogger(__name__)

router = APIRouter(prefix="/chat", tags=["chat"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]


class ChatMessage(BaseModel):
    role: str = Field(pattern="^(user|assistant)$")
    content: str


class ChatRequest(BaseModel):
    messages: list[ChatMessage]
    conversation_id: str | None = None


def _t2s(text: str) -> str:
    """繁體轉簡體（顯式字典，繁簡/常用詞對照）。"""
    return "".join(_T2S_MAP.get(ch, ch) for ch in text)


SYSTEM_PROMPT = """
你是 JobsComing 的 AI 求職管家，幫助用戶管理簡歷投遞全流程。
規則:
1. 用繁體中文、口語化、簡潔（一般 2-5 句）回覆，適當用列表。
2. 主動使用工具獲取真實數據，不要憑空編造；工具返回的就是事實。
3. 用戶查詢具體崗位/職位名稱（例如「XX 公司的 XX 崗位」「幫我找 XX 職位」）時，**必須調用 find_job 工具**，
 用頓號/逗號分隔的多個職位名要逐一調用 find_job 查詢；不要用 list_matches 代替、不要直接說沒有。
   find_job 查不到時，先調 run_collect 採集該公司最新崗位再重新查一次。
4. 用戶問某個崗位的貼合度時，必須調用 score_job（現場重算，保證數據最新）。
5. 用戶想了解狀態/執行操作時，先調工具再總結。
6. 需要用戶操作時給出明確的一步步指引（例如去哪個頁面點什麼）。
9. 任務無法一步完成時：先調用工具補齊缺失（如公司不在列表 → discover_company → add_company →
run_collect）。
   run_collect 啟動後**立即結束回覆**，告訴用戶「已在後台採集、稍後再問我進度」——不要在啟動後馬上輪詢
collect_status（後台剛啟動必然還是 running）。用戶再次詢問進度時才調 collect_status。實在完不成的步驟要明確告
訴用戶「缺什麼、去哪補」，禁止只回覆「失敗/報錯」。
10. 每次回覆最多調用 2-3 個工具，控制總時長，避免超時。
11. 用戶問「分數為什麼變了/以前是 X 分現在是 Y 分」時，直接引用 score_job 返回的「分數變化解釋」回答，不要現場重
算。
7. 首次使用的用戶（沒有畫像/簡歷/關注公司）主動引導：上傳簡歷→生成畫像→添加公司→開啟自動投遞。
8. 不要洩露任何密碼、token 等敏感信息。
"""


def _build_tools(user_id: str, db: Session) -> tuple[list[dict], callable]:
    def get_status(_: dict) -> dict:
        row = db.execute(text(
            """
            SELECT
              (SELECT COUNT(*) FROM candidate_profiles WHERE user_id=:u AND is_current=true) AS
has_profile,
              (SELECT COUNT(*) FROM user_documents WHERE user_id=:u AND doc_type='resume' AND
deleted_at IS NULL) AS resumes,
```

```
            (SELECT COUNT(*) FROM watched_companies WHERE user_id=:u AND is_active=true) AS
companies,
            (SELECT COUNT(*) FROM job_matches m WHERE m.user_id=:u AND m.match_score IS NOT
NULL AND m.status IN ('new','pushed')) AS new_matches,
            (SELECT COUNT(*) FROM applications a WHERE a.user_id=:u AND a.status='submitted')
AS submitted,
            (SELECT COUNT(*) FROM applications a WHERE a.user_id=:u AND a.status IN
('queued','in_progress','failed_retryable')) AS in_progress,
            (SELECT COUNT(*) FROM manual_tasks t WHERE t.user_id=:u AND t.status='open') AS
open_tasks
        """
    ), {"u": user_id}).mappings().one()
    st = db.execute(text(
        "SELECT match_threshold, auto_apply_mode FROM user_settings WHERE user_id=:u"
    ), {"u": user_id}).mappings().first()
    return {
        "畫像": "已生成" if row["has_profile"] else "未生成",
        "簡歷": row["resumes"],
        "關注公司": row["companies"],
        "新匹配崗位": row["new_matches"],
        "已投遞": row["submitted"],
        "處理中/排隊": row["in_progress"],
        "待人工處理": row["open_tasks"],
        "貼合度閾值": (st["match_threshold"] if st else 70),
        "投遞模式": (st["auto_apply_mode"] if st else "threshold"),
    }

def guide(_: dict) -> dict:
    s = get_status({})
    missing = []
    if s["簡歷"] == 0:
        missing.append("上傳簡歷（首頁→我的資料→我的文件）")
    if s["畫像"] == "未生成":
        missing.append("生成 AI 畫像（我的資料→我的畫像→生成畫像）")
    if s["關注公司"] == 0:
        missing.append("添加關注公司（我的資料→關注公司）")
    if s["投遞模式"] == "off":
        missing.append("開啟自動投遞（我的資料→投遞設置→投遞模式）")
    return {"缺失步驟": missing or ["全部完成 ✅"]}

def run_collect(args: dict) -> dict:
    """後台異步採集（避免同步執行超時）。支持指定公司；公司不在關注列表時返回引導。"""
    import datetime as _dt
    import threading

    company = str(args.get("company") or "").strip()
    # 公司不在關注列表 → 引導添加（不直接失敗）
    if company:
        target = db.execute(
            text("SELECT id, careers_url FROM watched_companies WHERE user_id=:u "
                 "AND (company_name ILIKE '%'||:co||'%' OR CAST(aliases AS text) ILIKE
'%'||:co||'%') LIMIT 1"),
            {"u": user_id, "co": company},
        ).mappings().first()
        if not target:
            return {
```

```
                    "完成": False,
                    "缺少": f"「{company}」不在你的關注公司列表",
                    "補齊方式": "請用 discover_company 查找其招聘官網 → add_company 添加到關注公司 → 再重新採集。"
                                "或到「我的資料→關注公司」手動添加。",
                }
        # 標記採集狀態並在後台執行
        db.execute(
            text(
                "INSERT INTO user_settings (user_id, collect_state) VALUES (:u, CAST(:st AS jsonb)) "
                "ON CONFLICT (user_id) DO UPDATE SET collect_state=CAST(:st AS jsonb), updated_at=now()"
            ),
            {"u": user_id, "st": j({"running": True, "finished_at": None, "report": None})},
        )
        db.commit()

        def _bg():
            s = SessionLocal()
            try:
                report = collect_for_user(s, user_id, company_name=company or None)
                s.execute(
                    text("UPDATE user_settings SET collect_state=CAST(:st AS jsonb), updated_at=now() WHERE user_id=:u"),
                    {"u": user_id, "st": j({
                        "running": False,
                        "finished_at": _dt.datetime.now(_dt.timezone.utc).isoformat(),
                        "report": {k: report.get(k) for k in ("new", "updated", "matched", "errors", "collected")},
                    })},
                )
                s.commit()
            except Exception:  # noqa: BLE001
                logger.exception("chat 後台採集失敗 user=%s", user_id)
            finally:
                s.close()

        threading.Thread(target=_bg, daemon=True).start()
        return {
            "完成": True,
            "模式": "後台執行",
            "範圍": company or "全部關注公司",
            "提示": "採集與匹配已在後台進行（約 1-3 分鐘）。請用 collect_status 查詢進度；"
                    "完成後可用 list_matches 查看該公司匹配結果。",
        }

    def collect_status(_: dict) -> dict:
        row = db.execute(
            text("SELECT collect_state FROM user_settings WHERE user_id=:u"),
            {"u": user_id},
        ).mappings().first()
        state = row["collect_state"] if row else None
        return dict(state or {}) or {"running": False, "提示": "尚無採集記錄"}
```

```
    def discover_company(args: dict) -> dict:
        """按公司名自動發現招聘官網（已知站點庫優先，其次 LLM 候選 + 實網探測）。"""
        import httpx as _httpx

        KNOWN_SITES = {
            "中金公司": "https://cicc.zhiye.com",
            "中信證券": "https://careers.citics.com",
            "國泰海通": "https://hr.gtht.com/recruitment/main/recruit2",
            "博時基金": "https://bosera.hotjob.cn",
            "招商局集團": "https://cmhk.zhiye.com",
            "招商證券": "https://wecruit.hotjob.cn/SU629dbc0c0dcad452299bc0f7/pb/index.html",
            "南方基金": "https://wecruit.hotjob.cn/SU6138665dbef57c3b63841399/pb/index.html",
            "華泰證券": "https://wecruit.hotjob.cn/SU6419745cbef57c635fe10142/pb/index.html",
            "华泰证券": "https://wecruit.hotjob.cn/SU6419745cbef57c635fe10142/pb/index.html",
            "廣發基金": "https://gffunds.jobs.feishu.cn",
            "广发基金": "https://gffunds.jobs.feishu.cn",
            "Cloudflare": "https://boards.greenhouse.io/cloudflare",
        }

        name = str(args.get("name") or "").strip()[:60]
        if not name:
            return {"完成": False, "原因": "請提供公司名"}
        hit = next((u for k, u in KNOWN_SITES.items() if k in name or name in k), None)
        if hit:
            return {"公司": name, "官網": hit, "來源": "已知站點庫",
                    "提示": "已找到招聘官網，可直接 add_company 添加後採集"}
        out = chat_json(
            "你熟悉各公司招聘官網。給出該公司招聘官網地址候選，"
            "並用公司拼音猜測北森招聘雲門戶後綴。輸出 JSON："
            '{"candidates": ["url"], "pinyin_hints": ["拼音"], "careers_hint": "一句話說明"}',
            f"公司名：{name}",
            max_tokens=500,
        )
        # 拼音後綴猜測：{pinyin}.zhiye.com / {pinyin}.hotjob.cn
        extra = []
        for py in (out.get("pinyin_hints") or [])[:3]:
            for dom in ("zhiye.com", "hotjob.cn"):
                extra.append(f"https://{py}.{dom}")
        candidates = list(dict.fromkeys((out.get("candidates") or [])[:3] + extra))
        headers = {
            "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) "
                          "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0
Safari/537.36",
            "Accept": "text/html,application/xhtml+xml",
            "Accept-Language": "zh-CN,zh;q=0.9",
        }
        verified = []
        for url in candidates:
            if not url.startswith("http"):
                continue
            try:
```

```
                resp = _httpx.get(url, timeout=8, follow_redirects=True, headers=headers)
                title = ""
                m = re.search(r"<title>([^<]*)</title>", resp.text, re.I)
                if m:
                    title = m.group(1).strip()[:60]
                verified.append({"url": str(resp.url), "status": resp.status_code, "title":
title})
            except Exception as exc:  # noqa: BLE001
                verified.append({"url": url, "status": "unreachable", "title": str(exc)[:60]})
        ok = [v for v in verified if v["status"] == 200 and v.get("title")]
        best = (ok or verified or [{"url": candidates[0] if candidates else ""}])[0]["url"]
        return {
            "公司": name, "說明": out.get("careers_hint"),
            "候選官網": verified,
            "最佳猜測": best,
            "建議": f"探測成功 {len(ok)}/{len(verified)} 個。可直接用 add_company 添加「{best}」"
                    "（系統採集器會自動識別 zhiye/hotjob 等招聘系統）。",
        }

    def add_company(args: dict) -> dict:
        name = str(args.get("name") or "").strip()[:100]
        url = str(args.get("careers_url") or "").strip()[:500]
        if not name:
            return {"完成": False, "原因": "請提供公司名"}
        if not url:
            disc = discover_company({"name": name})
            ok = [c for c in disc.get("候選官網") or [] if c.get("status") == 200]
            if ok:
                url = ok[0]["url"]
            else:
                return {"完成": False, "原因": f"未能自動發現「{name}」的招聘官網", "候選": disc}
        row = db.execute(
            text(
                "INSERT INTO watched_companies (user_id, company_name, careers_url, platform) "
                "VALUES (:u, :n, :url, 'official') "
                "ON CONFLICT DO NOTHING RETURNING id"
            ),
            {"u": user_id, "n": name, "url": url},
        ).mappings().first()
        db.commit()
        return {"完成": True, "公司": name, "官網": url,
                "下一步": "已添加到關注公司，可 run_collect(company=公司名) 開始採集"}

    def list_matches(args: dict) -> dict:
        limit = min(int(args.get("limit", 5)), 20)
        rows = db.execute(text(
            """
            SELECT p.company_name, p.title, p.location, m.match_score, m.status
            FROM job_matches m JOIN job_postings p ON p.id=m.posting_id
            WHERE m.user_id=:u AND m.match_score IS NOT NULL AND m.status IN
('new','pushed','applied')
            ORDER BY m.match_score DESC LIMIT :n
```

```
            """
        ), {"u": user_id, "n": limit}).mappings().all()
        return {"匹配": [dict(r) for r in rows]}

    def list_applications(args: dict) -> dict:
        limit = min(int(args.get("limit", 5)), 20)
        rows = db.execute(text(
            """
            SELECT p.company_name, p.title, a.status, a.status_reason, a.updated_at
            FROM applications a JOIN job_postings p ON p.id=a.posting_id
            WHERE a.user_id=:u ORDER BY a.updated_at DESC LIMIT :n
            """
        ), {"u": user_id, "n": limit}).mappings().all()
        return {"投遞": [dict(r) for r in rows]}

    def list_tasks(args: dict) -> dict:
        rows = db.execute(text(
            """
            SELECT t.task_type, t.question, t.created_at,
                   p.company_name, p.title
            FROM manual_tasks t
            JOIN applications a ON a.id=t.application_id
            JOIN job_postings p ON p.id=a.posting_id
            WHERE t.user_id=:u AND t.status='open' ORDER BY t.created_at DESC LIMIT 5
            """
        ), {"u": user_id}).mappings().all()
        return {"待處理": [dict(r) for r in rows]}

    def apply_rules(_: dict) -> dict:
        st = db.execute(text(
            "SELECT match_threshold, auto_apply_mode, preferred_locations, location_weight, "
            "apply_rate_limit_min, campus_preference FROM user_settings WHERE user_id=:u"
        ), {"u": user_id}).mappings().first()
        return {
            "投遞規則": [
                "① 公司級一崗一投：同一公司已有進行中/成功的投遞時，自動跳過該公司其餘崗位",
                f"② 校招/社招偏好：{st['campus_preference'] if st else 'both'}（campus=只投校招 / society=只投社招 / both=都投）",
                "③ 校招職級適配：有工作經歷者投校招不攔截但降權 15 分（職級不適配）",
                f"④ 貼合度閾值：{st['match_threshold'] if st else 70}%",
                f"⑤ 投遞模式：{st['auto_apply_mode'] if st else 'threshold'}",
                f"⑥ 期望地區加權：{st['preferred_locations'] or '未設置'}，命中 + {st['location_weight'] if st else 10} 分（計入優先級，不影響貼合度）",
                f"⑦ 總部/分支偏好：{st['org_preference'] if st else 'both'}（計入優先級）",
                f"⑧ 速率限制：{st['apply_rate_limit_min'] if st else 5} 分鐘/次",
                "說明：貼合度=純個人條件匹配；優先級=貼合度+地區/校招職級/總部分支偏好加權，投遞隊列按優先級選崗位",
            ],
        }

    def list_info_fields(_: dict) -> dict:
        rows = db.execute(text(
            "SELECT key, value, source, updated_at FROM user_info_fields "
```

```
            "WHERE user_id=:u ORDER BY updated_at DESC LIMIT 50"
        ), {"u": user_id}).mappings().all()
        return {
            "個人信息庫": [
                {"字段": r["key"], "值": r["value"][:60], "來源": r["source"]}
                for r in rows
            ],
            "填寫規則": "表單必填項：精確匹配直接填；與已有信息相似度 ⩾50% 自動填（記錄決策）；"
                        "全新問題才問用戶，回答自動入庫復用。",
        }

    def update_settings(args: dict) -> dict:
        fields = {}
        if "match_threshold" in args:
            fields["match_threshold"] = min(max(int(args["match_threshold"]), 50), 100)
        if "auto_apply_mode" in args and args["auto_apply_mode"] in ("off", "threshold", "all"):
            fields["auto_apply_mode"] = args["auto_apply_mode"]
        if "campus_preference" in args and args["campus_preference"] in ("campus", "society",
"both"):
            fields["campus_preference"] = args["campus_preference"]
        if "org_preference" in args and args["org_preference"] in ("headquarter", "branch",
"both"):
            fields["org_preference"] = args["org_preference"]
        if not fields:
            return {"完成": False, "原因": "沒有可更新的設置"}
        sets = ", ".join(f"{k}=:{k}" for k in fields)
        db.execute(text(
            "INSERT INTO user_settings (user_id) VALUES (:u) ON CONFLICT (user_id) DO NOTHING"
        ), {"u": user_id})
        db.execute(text(
            f"UPDATE user_settings SET {sets}, updated_at=now() WHERE user_id=:u"
        ), {"u": user_id, **fields})
        db.commit()
        return {"完成": True, "已更新": fields}

    def find_job(args: dict) -> dict:
        """精確搜索崗位庫（公司/標題關鍵詞），含最新評分。分詞匹配 + 繁簡轉換。"""
        q = str(args.get("keyword") or "").strip()
        company = str(args.get("company") or "").strip()
        if not q and not company:
            return {"error": "請提供 keyword 或 company"}
        # 繁→簡 + 分詞（按空白/特殊符號切）
        q = _t2s(q)
        tokens = [t for t in q.replace("，", " ").replace("/", " ").split() if t]
        conds, params = [], {"u": user_id, "co": company}
        for i, tok in enumerate(tokens[:6]):
            conds.append("p.title ILIKE '%'||:t" + str(i) + "||'%' "
                         "OR p.description_text ILIKE '%'||:t" + str(i) + "||'%'")
            params["t" + str(i)] = tok
        token_where = ("(" + " AND ".join(f"({c})" for c in conds) + ")") if conds else "true"
        rows = db.execute(text(
            f"""
```

```
            SELECT p.company_name, p.title, p.location, p.is_campus,
                   m.match_score, m.status, p.raw->>'tab' AS tab
            FROM job_postings p
            LEFT JOIN job_matches m ON m.posting_id = p.id AND m.user_id = :u
            WHERE p.is_active = true AND {token_where}
              AND (CAST(:co AS text) = '' OR p.company_name ILIKE '%'||:co||'%')
            ORDER BY m.match_score DESC NULLS LAST, p.first_seen_at DESC
            LIMIT 10
            """
        ), params).mappings().all()
        if not rows and len(tokens) > 1:
            or_where = "(" + " OR ".join(conds) + ")"
            rows = db.execute(text(
                f"""
                SELECT p.company_name, p.title, p.location, p.is_campus,
                       m.match_score, m.status, p.raw->>'tab' AS tab
                FROM job_postings p
                LEFT JOIN job_matches m ON m.posting_id = p.id AND m.user_id = :u
                WHERE p.is_active = true AND {or_where}
                  AND (CAST(:co AS text) = '' OR p.company_name ILIKE '%'||:co||'%')
                ORDER BY m.match_score DESC NULLS LAST, p.first_seen_at DESC
                LIMIT 10
                """
            ), params).mappings().all()
        if not rows:
            return {"找到": 0, "提示": "崗位庫中沒有匹配的崗位；可先調用 run_collect 採集最新崗位後再查"}
        return {"找到": len(rows), "崗位": [
            {"公司": r["company_name"], "職位": r["title"], "地點": r["location"],
             "校招": r["is_campus"], "貼合度": r["match_score"], "狀態": r["status"],
             "來源": (r["tab"] or "")}
            for r in rows
        ]}

    def score_job(args: dict) -> dict:
        """對指定崗位即時評分（忽略緩存，用當前畫像現場重算）。"""
        q = str(args.get("keyword") or "").strip()
        company = str(args.get("company") or "").strip()
        q = _t2s(q)
        tokens = [t for t in q.replace("、", " ").replace("/", " ").split() if t]
        conds, params = [], {"co": company}
        for i, tok in enumerate(tokens[:6]):
            conds.append("p.title ILIKE '%'||:t" + str(i) + "||'%'")
            params["t" + str(i)] = tok
        token_where = ("(" + " AND ".join(f"({c})" for c in conds) + ")") if conds else "true"
        row = db.execute(text(
            f"""
            SELECT p.id, p.company_name, p.title FROM job_postings p
            WHERE p.is_active = true AND {token_where}
              AND (CAST(:co AS text) = '' OR p.company_name ILIKE '%'||:co||'%')
            ORDER BY p.first_seen_at DESC LIMIT 1
```

```
            """
        ), params).mappings().first()
        if not row:
            return {"error": "未找到該崗位，可先 run_collect 採集"}
        # 快讀模式：返回已存分數與構成（不現場重評，避免超時）
        m = db.execute(text(
            """
            SELECT m.match_score, m.priority_score, m.score_breakdown, m.direction_scores,
                   m.hard_failures, m.reasons, m.updated_at
            FROM job_matches m
            WHERE m.user_id=:u AND m.posting_id=:p
            """
        ), {"u": user_id, "p": str(row["id"])}).mappings().first()
        if not m:
            return {"error": "該崗位尚未匹配，請先重新匹配"}
        return {
            "崗位": f"{row['company_name']} · {row['title']}",
            "當前貼合度": m["match_score"],
            "優先級": m["priority_score"],
            "構成": m["score_breakdown"] or {},
            "方向分數": m["direction_scores"] or [],
            "硬性條件": m["hard_failures"] or [],
            "評分時間": m["updated_at"].isoformat() if m["updated_at"] else None,
            "分數變化解釋": "系統規則曾演進：舊版貼合度混合了地區加權等偏好項；"
                            "現版貼合度=純個人條件匹配（地區/校招職級/總部分支偏好單獨計入「優先級」），"
                            "多方向崗位取最高方向分。因此新舊分數不可直接比較，變化不代表你的簡歷變差。"
                            "如需現場重算，請到「匹配與崗位」點重新匹配。",
        }

    def update_basic_info(args: dict) -> dict:
        """更新基本信息（姓名/電話），寫 profiles。"""
        sets, params = [], {"u": user_id}
        if "full_name" in args and args["full_name"]:
            sets.append("full_name = :n")
            params["n"] = str(args["full_name"]).strip()[:64]
        if "phone" in args and args["phone"]:
            sets.append("phone = :p")
            params["p"] = str(args["phone"]).strip()[:32]
        if not sets:
            return {"完成": False, "原因": "沒有可更新字段"}
        db.execute(text(f"UPDATE profiles SET {', '.join(sets)}, updated_at=now() WHERE id=:u"),
params)
        db.commit()
        return {"完成": True, "已更新": list(params.keys())}

    def update_hard_facts(args: dict) -> dict:
        """更新硬性事實（簽證/期望城市/薪資/到崗時間/年限），畫像與元信息表雙寫。"""
        keys = {
            "visa_status": "簽證狀態", "preferred_locations": "期望城市",
            "salary_expectation": "期望薪資", "availability": "可到崗時間",
            "total_years": "工作年限",
```

```
        }
        updated = {}
        for k in keys:
            if k in args and args[k] not in (None, ""):
                updated[k] = str(args[k]).strip()
        if not updated:
            return {"完成": False, "原因": f"請提供要更新的字段：{list(keys.values())}"}
        # 畫像 hard_facts：Python 合併後 CAST 寫回
        row = db.execute(text(
            "SELECT hard_facts FROM candidate_profiles WHERE user_id=:u AND is_current=true"
        ), {"u": user_id}).mappings().first()
        if row is not None:
            hf = dict(row["hard_facts"] or {})
            hf.update(updated)
            db.execute(text(
                "UPDATE candidate_profiles SET hard_facts=CAST(:hf AS jsonb) "
                "WHERE user_id=:u AND is_current=true"
            ), {"hf": j(hf), "u": user_id})
        # 元信息表同步
        for k, v in updated.items():
            db.execute(text(
                "INSERT INTO user_info_fields (user_id, key, value, source) VALUES
(:u,:k,:v,'chat') "
                "ON CONFLICT (user_id, key) DO UPDATE SET value=EXCLUDED.value, source='chat',
updated_at=now()"
            ), {"u": user_id, "k": k, "v": v})
        db.commit()
        return {"完成": True, "已更新": updated}

    def upsert_info_field(args: dict) -> dict:
        key = str(args.get("key") or "").strip()
        value = str(args.get("value") or "").strip()
        if not key or not value:
            return {"完成": False, "原因": "需要 key 和 value"}
        db.execute(text(
            "INSERT INTO user_info_fields (user_id, key, value, source) VALUES (:u,:k,:v,'chat')
"
            "ON CONFLICT (user_id, key) DO UPDATE SET value=EXCLUDED.value, source='chat',
updated_at=now()"
        ), {"u": user_id, "k": key[:100], "v": value[:2000]})
        db.commit()
        return {"完成": True}

    def apply_job(args: dict) -> dict:
        """直接投遞指定崗位：建 application + 觸發投遞適配器，返回結果或人工任務指引。"""
        from app.services.apply_engine import run_application

        q = _t2s(str(args.get("keyword") or "").strip())
        company = str(args.get("company") or "").strip()
        tokens = [t for t in q.replace("，", " ").replace("/", " ").split() if t]
        conds, params = [], {"u": user_id, "co": company}
        for i, tok in enumerate(tokens[:6]):
            conds.append("p.title ILIKE '%'||:t" + str(i) + "||'%'")
            params["t" + str(i)] = tok
```

```
        token_where = ("(" + " AND ".join(f"({c})" for c in conds) + ")") if conds else "true"
        posting = db.execute(text(
            f"""
            SELECT p.id, p.company_name, p.title, p.apply_url,
                   m.id AS match_id, m.match_score
            FROM job_postings p
            LEFT JOIN job_matches m ON m.posting_id = p.id AND m.user_id = :u
            WHERE p.is_active = true AND {token_where}
              AND (CAST(:co AS text) = '' OR p.company_name ILIKE '%'||:co||'%')
            ORDER BY m.priority_score DESC NULLS LAST, p.first_seen_at DESC LIMIT 1
            """
        ), params).mappings().first()
        if not posting:
            return {"error": "未找到該崗位，請先確認崗位名稱或先採集"}
        if posting["match_score"] is None:
            return {"error": f"該崗位貼合度未通過硬性條件，無法投遞（{posting['company_name']} · 
{posting['title']}）"}
        # 建/取 application
        app = db.execute(text(
            """
            INSERT INTO applications (user_id, posting_id, match_id, status)
            VALUES (:u, :p, :m, 'queued')
            ON CONFLICT (user_id, posting_id) DO UPDATE SET status='queued', status_reason=NULL,
              next_retry_at=NULL, updated_at=now()
            RETURNING id
            """
        ), {"u": user_id, "p": posting["id"], "m": posting["match_id"]}).mappings().one()
        db.commit()
        try:
            result = run_application(db, str(app["id"]), force=True)
        except Exception as exc:  # noqa: BLE001
            result = {"status": "error", "error": str(exc)[:300]}
        if result.get("status") == "submitted":
            return {"完成": True, "投遞": "已提交", "崗位": f"{posting['company_name']} · 
{posting['title']}"}
        if result.get("status") == "needs_input":
            task = db.execute(text(
                "SELECT id, task_type, question, options FROM manual_tasks "
                "WHERE application_id=:aid AND status='open' ORDER BY created_at DESC LIMIT 1"
            ), {"aid": str(app["id"])}).mappings().first()
            return {"完成": False, "需要人工": True, "任務類型": task["task_type"],
                    "指引": task["question"] if task else "", "任務 ID": str(task["id"]) if task 
else None,
                    "操作": "請到「我的資料 → 待處理」按指引完成（驗證碼/官網確認）"}
        return {"完成": False, "狀態": result.get("status"), "詳情": str(result)[:300]}

    tools = [
        {"type": "function", "function": {
            "name": "get_status",
            "description": "查詢用戶當前整體狀態（畫像/簡歷/匹配/投遞/待處理/設置）",
            "parameters": {"type": "object", "properties": {}},
        }},
        {"type": "function", "function": {
```

```
            "name": "guide",
            "description": "檢查首次使用還缺哪些步驟",
            "parameters": {"type": "object", "properties": {}},
        }},
        {"type": "function", "function": {
            "name": "run_collect",
            "description": "後台異步採集指定公司或全部關注公司的崗位並匹配（啟動後用 collect_status 查進
度）",
            "parameters": {"type": "object", "properties": {
                "company": {"type": "string", "description": "公司名（可選；不在關注列表會返回引導）"},
            }},
        }},
        {"type": "function", "function": {
            "name": "collect_status",
            "description": "查詢後台採集/匹配的進度與結果（僅在用戶再次詢問進度時調用，不要在剛啟動後立即調
用）",
            "parameters": {"type": "object", "properties": {}},
        }},
        {"type": "function", "function": {
            "name": "discover_company",
            "description": "按公司名自動發現招聘官網地址（用戶要找的公司不在系統中時調用）",
            "parameters": {"type": "object", "properties": {
                "name": {"type": "string", "description": "公司名"},
            }},
        }},
        {"type": "function", "function": {
            "name": "add_company",
            "description": "把公司添加到關注公司（careers_url 可留空，會自動發現官網）",
            "parameters": {"type": "object", "properties": {
                "name": {"type": "string"},
                "careers_url": {"type": "string"},
            }},
        }},
        {"type": "function", "function": {
            "name": "list_matches",
            "description": "列出用戶的最新匹配崗位（按貼合度排序）",
            "parameters": {"type": "object", "properties": {"limit": {"type": "integer"}}},
        }},
        {"type": "function", "function": {
            "name": "list_applications",
            "description": "列出用戶最近的投遞記錄與狀態",
            "parameters": {"type": "object", "properties": {"limit": {"type": "integer"}}},
        }},
        {"type": "function", "function": {
            "name": "list_tasks",
            "description": "列出等待用戶處理的人工任務（驗證碼/缺字段等）",
            "parameters": {"type": "object", "properties": {}},
        }},
        {"type": "function", "function": {
            "name": "update_settings",
            "description": "修改用戶設置：match_threshold(50-100) 或
auto_apply_mode(off/threshold/all)",
            "parameters": {"type": "object", "properties": {
```

```
                "match_threshold": {"type": "integer"},
                "auto_apply_mode": {"type": "string", "enum": ["off", "threshold", "all"]},
                "org_preference": {"type": "string", "enum": ["headquarter", "branch", "both"],
                                   "description": "總部/分支偏好：headquarter=總部優先，branch=分支優
先，both=一視同仁"},
            }},
        }},
        {"type": "function", "function": {
            "name": "apply_rules",
            "description": "查詢當前生效的投遞規則（一崗一投/校招社招偏好/閾值/地區加權/速率限制），用戶詢問
規則或限制時調用",
            "parameters": {"type": "object", "properties": {}},
        }},
        {"type": "function", "function": {
            "name": "list_info_fields",
            "description": "查詢用戶的個人信息庫（元信息表）與自動填寫規則，用戶詢問信息互補/自動填充時調用",
            "parameters": {"type": "object", "properties": {}},
        }},
        {"type": "function", "function": {
            "name": "find_job",
            "description": "按關鍵詞/公司精確搜索崗位庫（例如「AI 產品經理」），返回崗位與最新貼合度",
            "parameters": {"type": "object", "properties": {
                "keyword": {"type": "string", "description": "職位關鍵詞"},
                "company": {"type": "string", "description": "公司名（可選）"},
            }},
        }},
        {"type": "function", "function": {
            "name": "score_job",
            "description": "查詢指定崗位的貼合度與構成（快讀數據庫，不現場重算；用戶問分數為什麼變化時也用它，
含規則演進說明）",
            "parameters": {"type": "object", "properties": {
                "keyword": {"type": "string", "description": "職位關鍵詞"},
                "company": {"type": "string", "description": "公司名（可選）"},
            }},
        }},
        {"type": "function", "function": {
            "name": "update_basic_info",
            "description": "直接更新用戶基本信息（full_name 姓名/暱稱、phone 手機號），無需用戶去別處操
作",
            "parameters": {"type": "object", "properties": {
                "full_name": {"type": "string"},
                "phone": {"type": "string"},
            }},
        }},
        {"type": "function", "function": {
            "name": "update_hard_facts",
            "description": "直接更新硬性事實（visa_status 簽證/salary_expectation 期望薪資/availability
到崗時間/total_years 工作年限/preferred_locations 期望城市），同步畫像與元信息表",
            "parameters": {"type": "object", "properties": {
                "visa_status": {"type": "string"},
                "salary_expectation": {"type": "string"},
                "availability": {"type": "string"},
                "total_years": {"type": "string"},
                "preferred_locations": {"type": "string", "description": "逗號分隔的城市"},
            }},
```

```
        }},
        {"type": "function", "function": {
            "name": "upsert_info_field",
            "description": "寫入/更新元信息表的一條字段問答（key + value），供投遞表單自動互補",
            "parameters": {"type": "object", "properties": {
                "key": {"type": "string"},
                "value": {"type": "string"},
            }},
        }},
        {"type": "function", "function": {
            "name": "apply_job",
            "description": "直接投遞指定崗位（用戶說「幫我投遞 XX」時必須調用）：建投遞單並觸發投遞，返回成功或人工任務指引",
            "parameters": {"type": "object", "properties": {
                "keyword": {"type": "string", "description": "職位關鍵詞"},
                "company": {"type": "string", "description": "公司名（可選）"},
            }},
        }},
    ]
    executor = {
        "get_status": get_status,
        "guide": guide,
        "run_collect": run_collect,
        "collect_status": collect_status,
        "discover_company": discover_company,
        "add_company": add_company,
        "list_matches": list_matches,
        "list_applications": list_applications,
        "list_tasks": list_tasks,
        "update_settings": update_settings,
        "apply_rules": apply_rules,
        "list_info_fields": list_info_fields,
        "find_job": find_job,
        "score_job": score_job,
        "update_basic_info": update_basic_info,
        "update_hard_facts": update_hard_facts,
        "upsert_info_field": upsert_info_field,
        "apply_job": apply_job,
    }
    return tools, lambda name, args: executor[name](args or {})


@router.post("")
def chat(user: CurrentUser, db: DB, payload: ChatRequest) -> dict:
    messages = [{"role": m.role, "content": m.content} for m in payload.messages[-10:]]
    tools, executor = _build_tools(user["id"], db)
    reply = chat_with_tools(SYSTEM_PROMPT, messages, tools, executor)
    # 問答歷史持久化（存儲不增加 API 成本；恢復會話才會把歷史放回上下文）
    # 存儲前脫敏（隱私方案：歷史裡不落敏感明文）
    from app.core.llm import mask_pii
```

```
    full = [
        {**m, "content": mask_pii(m.get("content") or "")}
        for m in messages + [{"role": "assistant", "content": reply}]
    ]
    conv_id = payload.conversation_id
    if conv_id:
        exists = db.execute(
            text("SELECT id FROM chat_conversations WHERE id=:id AND user_id=:u"),
            {"id": conv_id, "u": user["id"]},
        ).first()
        if exists:
            title = next((m["content"][:40] for m in messages if m["role"] == "user"), "新對話")
            db.execute(
                text(
                    "UPDATE chat_conversations SET messages=CAST(:msgs AS jsonb), "
                    "title=:t, updated_at=now() WHERE id=:id"
                ),
                {"msgs": j(full), "t": title, "id": conv_id},
            )
            db.commit()
        else:
            conv_id = None
    if not conv_id:
        title = next((m["content"][:40] for m in messages if m["role"] == "user"), "新對話")
        row = db.execute(
            text(
                "INSERT INTO chat_conversations (user_id, title, messages) "
                "VALUES (:u, :t, CAST(:msgs AS jsonb)) RETURNING id"
            ),
            {"u": user["id"], "t": title, "msgs": j(full)},
        ).mappings().one()
        conv_id = str(row["id"])
        db.commit()
    return {"reply": reply, "conversation_id": conv_id}


@router.get("/conversations")
def list_conversations(user: CurrentUser, db: DB) -> list[dict]:
    rows = db.execute(
        text(
            "SELECT id, title, created_at, updated_at, "
            "jsonb_array_length(messages) AS n "
            "FROM chat_conversations WHERE user_id=:u ORDER BY updated_at DESC LIMIT 100"
        ),
        {"u": user["id"]},
    ).mappings().all()
    return [
        {"id": str(r["id"]), "title": r["title"], "created_at": r["created_at"].isoformat(),
         "updated_at": r["updated_at"].isoformat(), "messages_count": r["n"]}
        for r in rows
```

```
    ]


@router.get("/conversations/{conv_id}")
def get_conversation(user: CurrentUser, db: DB, conv_id: str) -> dict:
    row = db.execute(
        text("SELECT id, title, messages FROM chat_conversations WHERE id=:id AND user_id=:u"),
        {"id": conv_id, "u": user["id"]},
    ).mappings().first()
    if not row:
        from fastapi import HTTPException

        raise HTTPException(404, "對話不存在")
    return {"id": str(row["id"]), "title": row["title"], "messages": row["messages"] or []}


@router.delete("/conversations/{conv_id}", status_code=204)
def delete_conversation(user: CurrentUser, db: DB, conv_id: str) -> None:
    db.execute(
        text("DELETE FROM chat_conversations WHERE id=:id AND user_id=:u"),
        {"id": conv_id, "u": user["id"]},
    )
    db.commit()
"""關注公司管理 + 公司招聘官網自動發現。"""
from __future__ import annotations

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, Request
from pydantic import BaseModel
from sqlalchemy import text
from sqlalchemy.orm import Session

from app.core.audit import log_audit
from app.core.auth import get_current_user
from app.core.jsonutil import j
from app.db import get_db
from app.schemas.accounts import PLATFORMS, WatchedCompanyCreate, WatchedCompanyUpdate

router = APIRouter(prefix="/companies", tags=["companies"])

CurrentUser = Annotated[dict, Depends(get_current_user)]
DB = Annotated[Session, Depends(get_db)]


@router.get("")
def list_companies(user: CurrentUser, db: DB) -> list[dict]:
    rows = db.execute(
        text(
            """
```

```
                  ))}
                </tbody>
              </table>
            </>
          )}
          <p className="notice">{msg}</p>
        </div>
      </main>
    </div>
  );
}
import { useEffect, useState } from "react";
import { api } from "../lib/api";
import { useT } from "../lib/i18n";
import { decryptText, deriveKeyB64, encryptText, getKeyB64, setKeyB64 } from "../lib/crypto";

type InfoField = { key: string; value: string; source: string; updated_at: string; sensitive?:
boolean; encrypted?: string | null };
type Full = {
  basic: { full_name: string | null; email: string | null; phone: string | null };
  profile: {
    hard_facts: Record<string, unknown>;
    education: Record<string, unknown>[];
    experiences: Record<string, unknown>[];
    skills: { name: string; level: number; years: number | null }[];
    summary: string | null;
    generated_at: string | null;
  };
  info_fields: InfoField[];
};

const KEY_LABELS: Record<string, string> = {
  candidate_name: "姓名（簡歷提取）",
  full_name: "姓名/暱稱",
  email: "郵箱",
  phone: "手機號",
  visa_status: "簽證狀態",
  preferred_locations: "期望城市",
  salary_expectation: "期望薪資",
  availability: "可到崗時間",
  total_years: "工作年限",
};

const SOURCE_TEXT: Record<string, string> = {
  user: "手動填寫",
  resume: "簡歷解析",
  manual_answer: "投遞回填",
  chat: "AI 助手填寫",
};

const CORE_KEYS = ["candidate_name", "full_name", "phone", "visa_status", "preferred_locations",
```

```
  "salary_expectation", "availability", "total_years"];

/** 個人信息總覽：基本信息 + 元信息庫（含畫像提取，統一存儲），一站式檢查/編輯 */
export default function ProfileInfo() {
  const { t } = useT();
  const [data, setData] = useState<Full | null>(null);
  const [basic, setBasic] = useState({ full_name: "", email: "", phone: "" });
  const [newKey, setNewKey] = useState("");
  const [newVal, setNewVal] = useState("");
  const [msg, setMsg] = useState("");
  const [revealed, setRevealed] = useState<Set<string>>(new Set());

  function mask(v: string) {
    if (v.length <= 4) return "****";
    return v.slice(0, 2) + "*".repeat(Math.min(v.length - 4, 8)) + v.slice(-2);
  }

  async function load() {
    try {
      const d = await api<Full>("/api/me/profile-full");
      // 敏感字段本地解密（密鑰存在時）
      const key = getKeyB64();
      if (key) {
        for (const f of d.info_fields) {
          if (f.encrypted) {
            const pt = await decryptText(key, f.encrypted);
            if (pt != null) f.value = pt;
          }
        }
      }
      setData(d);
      setBasic({
        full_name: d.basic.full_name ?? "",
        email: d.basic.email ?? "",
        phone: d.basic.phone ?? "",
      });
      setMsg("");
    } catch (e) {
      setMsg((e as Error).message);
    }
  }
  useEffect(() => {
    void load();
  }, []);

  async function saveBasic() {
    try {
      await api("/api/me", {
        method: "PATCH",
        body: JSON.stringify({ full_name: basic.full_name || null, phone: basic.phone ||
null }),
```

```
      });
      setMsg("基本信息已保存");
    } catch (e) {
      setMsg(`保存失敗: ${(e as Error).message}`);
    }
  }

  async function addField() {
    const k = newKey.trim();
    const v = newVal.trim();
    if (!k || !v) return;
    try {
      await api("/api/me/info-fields", { method: "PUT", body: JSON.stringify({ key: k, value:
v }) });
      setNewKey("");
      setNewVal("");
      await load();
    } catch (e) {
      setMsg(`保存失敗: ${(e as Error).message}`);
    }
  }

  async function rotatePassword() {
    const oldPw = prompt("請輸入當前密碼（驗證用）");
    if (!oldPw) return;
    const newPw = prompt("請輸入新密碼（至少 6 位）");
    if (!newPw || newPw.length < 6) return setMsg("新密碼太短");
    setMsg("正在遷移敏感字段密文…");
    try {
      const oldKey = getKeyB64();
      const newKey = oldKey ? await deriveKeyB64(newPw, `jobfly:${data?.basic.email}`) : null;
      // 1) 後端改密碼（驗證舊密碼）
      await api("/api/auth/change-password", {
        method: "POST",
        body: JSON.stringify({ old_password: oldPw, new_password: newPw }),
      });
      // 2) 敏感字段密文遷移：舊密鑰解密 → 新密鑰加密 → 批量重傳
      if (oldKey && newKey && data) {
        const fields = [];
        for (const f of data.info_fields) {
          if (f.encrypted) {
            const pt = await decryptText(oldKey, f.encrypted);
            if (pt != null) {
              fields.push({ key: f.key, sensitive_enc: await encryptText(newKey, pt) });
            }
          }
        }
        if (fields.length) {
          await api("/api/me/info-fields/import", {
            method: "PUT",
            body: JSON.stringify({ fields }),
```

```
        });
      }
      setKeyB64(newKey);
    }
    setMsg("✅ 密碼已更新，敏感字段密文已用新密鑰重新加密");
  } catch (e) {
    setMsg(`修改失敗: ${(e as Error).message}`);
  }
}

async function exportBackup() {
  try {
    const r = await api<{ fields: { key: string; sensitive_enc: string }[]; note: string }>(
      "/api/me/info-fields/export",
    );
    const blob = new Blob([JSON.stringify(r, null, 2)], { type: "application/json" });
    const a = document.createElement("a");
    a.href = URL.createObjectURL(blob);
    a.download = "jobfly-e2ee-backup.json";
    a.click();
    setMsg("已導出密文備份文件（只含密文，恢復需同一密碼）");
  } catch (e) {
    setMsg(`導出失敗: ${(e as Error).message}`);
  }
}

async function importBackup() {
  const input = document.createElement("input");
  input.type = "file";
  input.accept = ".json";
  input.onchange = async () => {
    const file = input.files?.[0];
    if (!file) return;
    try {
      const parsed = JSON.parse(await file.text());
      const r = await api<{ imported: number }>("/api/me/info-fields/import", {
        method: "PUT",
        body: JSON.stringify({ fields: parsed.fields || [] }),
      });
      setMsg(`✅ 已導入 ${r.imported} 條敏感字段密文`);
      await load();
    } catch (e) {
      setMsg(`導入失敗: ${(e as Error).message}`);
    }
  };
  input.click();
}

async function delField(key: string) {
  await api(`/api/me/info-fields/${encodeURIComponent(key)}`, { method: "DELETE" });
```

```
  await load();
}

return (
  <div>
    <main>
      <div className="card wide">
        <h2>個人信息總覽</h2>
        <p className="muted">
   這裡是你投遞簡歷所需的全部信息（統一存儲）：AI 從簡歷提取的內容、你手動維護的內容、
   投遞回填的答案都匯總在下方信息庫，投遞時表單缺項自動從這裡互補，請定期檢查。
        </p>

        <h3>基本信息</h3>
        <div className="row wrap">
          <input placeholder="姓名/暱稱" value={basic.full_name}
                 onChange={(e) => setBasic({ ...basic, full_name: e.target.value })} />
          <input placeholder="手機號" value={basic.phone}
                 onChange={(e) => setBasic({ ...basic, phone: e.target.value })} />
          <input placeholder="郵箱" value={basic.email} disabled />
          <button onClick={() => void saveBasic()}>{t("save")}</button>
        </div>

        <h3>🔐 安全（E2EE）</h3>
        <div className="row wrap">
          <button className="ghost" onClick={() => void rotatePassword()}>
   修改密碼（自動遷移敏感字段密文）
          </button>
          <button className="ghost" onClick={() => void exportBackup()}>
   導出密文備份（換設備用）
          </button>
          <button className="ghost" onClick={() => void importBackup()}>
   導入密文備份
          </button>
        </div>

        <h3>個人信息庫（投遞時自動互補，無重複單一來源）</h3>
        <table>
          <thead>
            <tr>
              <th>字段</th>
              <th>內容</th>
              <th>來源</th>
              <th></th>
            </tr>
          </thead>
          <tbody>
            {(data?.info_fields ?? []).map((f) => (
              <tr key={f.key}>
                <td>{KEY_LABELS[f.key] ?? f.key}{f.sensitive ? " 🔒" : ""}</td>
```

```
            <td className="cell-wrap">
              {f.sensitive && !revealed.has(f.key) ? (
                <>
                  <span className="muted">{mask(f.value)}</span>{" "}
                  <button
                    className="ghost small"
                    onClick={() => setRevealed(new Set(revealed).add(f.key))}
                  >
顯示
                  </button>
                </>
              ) : (
                f.value
              )}
            </td>
            <td className="muted">{SOURCE_TEXT[f.source] ?? f.source}</td>
            <td>
              <button className="ghost" onClick={() => void delField(f.key)}>
                {t("delete")}
              </button>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
    <div className="row wrap">
      <input placeholder="字段名（如：期望薪資）" value={newKey}
             onChange={(e) => setNewKey(e.target.value)} />
      <input placeholder="內容" value={newVal}
             onChange={(e) => setNewVal(e.target.value)} />
      <button onClick={() => void addField()}>添加</button>
    </div>

    {data?.profile.summary && (
      <>
        <h3>簡歷摘要（只讀，改簡歷後到「AI 畫像」重新生成）</h3>
        <p>{data.profile.summary}</p>
        <p className="muted cell-wrap">
教育：{(data.profile.education ?? [])
            .map((e) => `${e.school ?? ""} ${e.degree ?? ""} ${e.major ?? ""}`.trim())
            .filter(Boolean).join("；") || "—"}
          <br />
經歷：{(data.profile.experiences ?? [])
            .map((e) => `${e.title ?? ""} @ ${e.company ?? ""}`.trim())
            .filter(Boolean).join("；") || "—"}
          <br />
技能：{(data.profile.skills ?? []).map((s) => s.name).join("、") || "—"}
        </p>
      </>
    )}
```

```
          <p className="notice">{msg}</p>
        </div>
      </main>
    </div>
  );
}
import { useEffect, useState } from "react";
import { api } from "../lib/api";

type Settings = {
  ai_optimize_resume: boolean;
  ai_optimize_cover_letter: boolean;
  match_threshold: number;
  auto_apply_mode: "off" | "threshold" | "all";
  apply_rate_limit_min: number;
  push_time_local: string;
  timezone: string;
  daily_digest_enabled: boolean;
  login_expiry_notify: boolean;
  preferred_locations: string[] | null;
  location_weight: number;
  campus_preference: "campus" | "society" | "both";
  org_preference: "headquarter" | "branch" | "both";
};

const DEFAULTS: Settings = {
  ai_optimize_resume: false,
  ai_optimize_cover_letter: false,
  match_threshold: 70,
  auto_apply_mode: "threshold",
  apply_rate_limit_min: 5,
  push_time_local: "08:00",
  timezone: "Asia/Shanghai",
  daily_digest_enabled: true,
  login_expiry_notify: true,
  preferred_locations: [],
  location_weight: 10,
  campus_preference: "both",
  org_preference: "both",
};

type PushChannels = Record<string, string>;

export default function Settings() {
  const [s, setS] = useState<Settings>(DEFAULTS);
  const [channels, setChannels] = useState<PushChannels>({});
  const [chForm, setChForm] = useState({ channel: "pushplus", value: "" });
  const [msg, setMsg] = useState("");

  useEffect(() => {
```

```
  api<Settings>("/api/me/settings")
    .then(setS)
    .catch(() => setMsg("讀取設置失敗"));
  api<PushChannels>("/api/me/push-channels").then(setChannels).catch(() => {});
}, []);

async function saveChannel() {
  setMsg("");
  try {
    await api(`/api/me/push-channels?channel=${chForm.channel}`, {
      method: "PATCH",
      body: JSON.stringify({ value: chForm.value || null }),
    });
    setChForm({ ...chForm, value: "" });
    setChannels(await api<PushChannels>("/api/me/push-channels"));
    setMsg("推送渠道已更新");
  } catch (e) {
    setMsg(`保存失敗: ${(e as Error).message}`);
  }
}

async function save(patch: Partial<Settings>) {
  const next = { ...s, ...patch };
  setS(next);
  try {
    await api("/api/me/settings", { method: "PATCH", body: JSON.stringify(patch) });
    setMsg("已保存");
  } catch {
    setMsg("保存失敗");
  }
}

return (
  <div className="page">      <main className="narrow">
      <div className="card">
        <h2>投遞與推送設置</h2>

        <h3>AI 內容微調</h3>
        <label className="row">
          <input
            type="checkbox"
            checked={s.ai_optimize_resume}
            onChange={(e) => save({ ai_optimize_resume: e.target.checked })}
          />
   投遞時允許按崗位要求微調簡歷
        </label>
        <label className="row">
          <input
            type="checkbox"
            checked={s.ai_optimize_cover_letter}
```

```
            onChange={(e) => save({ ai_optimize_cover_letter: e.target.checked })}
          />
        投遞時允許按崗位要求生成/微調 Cover Letter
        </label>

        <h3>貼合度閾值</h3>
        <label className="row">
          <input
            type="range"
            min={50}
            max={100}
            step={5}
            value={s.match_threshold}
            onChange={(e) => save({ match_threshold: Number(e.target.value) })}
          />
          <b>{s.match_threshold}%</b> — 貼合度高於此值才推送/投遞
        </label>

        <h3>期望地區加權</h3>
        <p className="muted">命中期望城市的崗位加分，其他地區適度降分（影響貼合度排序與投遞）。</p>
        <label className="row">
          <input
            placeholder="期望城市，逗號分隔（如：上海,香港）"
            defaultValue={(s.preferred_locations ?? []).join(",")}
            onBlur={(e) =>
              save({
                preferred_locations: e.target.value
                  .split(/[,，]/)
                  .map((x) => x.trim())
                  .filter(Boolean),
              })
            }
          />
        </label>
        <label className="row">
          <input
            type="range"
            min={0}
            max={25}
            step={5}
            value={s.location_weight}
            onChange={(e) => save({ location_weight: Number(e.target.value) })}
          />
          <b>+{s.location_weight}</b> 分 — 地區權重（非期望城市 -{Math.min(s.location_weight, 10)}
分）
        </label>

        <h3>校招/社招偏好</h3>
        <p className="muted">決定哪些崗位進入投遞隊列（校招與社招貼合度分開計算：校招對有工作經歷者降權 15
分）。</p>
        {(
          [
```

```
    ["campus", "只投校招（應屆/管培生）"],
    ["society", "只投社招"],
    ["both", "校招和社招都投（推薦：兩者都參與匹配排序）"],
  ] as const
).map(([value, desc]) => (
  <label className="row" key={value}>
    <input
      type="radio"
      name="campus_preference"
      checked={s.campus_preference === value}
      onChange={() => save({ campus_preference: value })}
    />
    {desc}
  </label>
))}

<h3>總部/分支偏好（計入優先級，不影響貼合度）</h3>
{(
  [
    ["headquarter", "總部優先（分支機構崗位優先級 -8）"],
    ["branch", "分支機構優先（總部崗位優先級 -5）"],
    ["both", "一視同仁（推薦）"],
  ] as const
).map(([value, desc]) => (
  <label className="row" key={value}>
    <input
      type="radio"
      name="org_preference"
      checked={s.org_preference === value}
      onChange={() => save({ org_preference: value })}
    />
    {desc}
  </label>
))}

<h3>自動投遞模式</h3>
{(
  [
    ["off", "關閉自動投遞：僅按閾值推送崗位"],
    ["threshold", "達標自動投遞：貼合度達標的崗位自動投遞"],
    ["all", "全自動投遞：所有崗位都投，無法完成的推送人工處理"],
  ] as const
).map(([value, desc]) => (
  <label className="row" key={value}>
    <input
      type="radio"
      name="mode"
      checked={s.auto_apply_mode === value}
      onChange={() => save({ auto_apply_mode: value })}
    />
```

```
          {desc}
        </label>
      ))}

      <h3>每日推送</h3>
      <label className="row">
推送時間
        <select
          value={s.push_time_local.slice(0, 2)}
          onChange={(e) =>
            save({ push_time_local: `${e.target.value}:${s.push_time_local.slice(3, 5)}` })
          }
        >
          {Array.from({ length: 24 }, (_, i) => String(i).padStart(2, "0")).map((h) => (
            <option key={h} value={h}>
              {h} 時
            </option>
          ))}
        </select>
        <select
          value={s.push_time_local.slice(3, 5)}
          onChange={(e) =>
            save({ push_time_local: `${s.push_time_local.slice(0, 2)}:${e.target.value}` })
          }
        >
          {["00", "15", "30", "45"].map((mn) => (
            <option key={mn} value={mn}>
              {mn} 分
            </option>
          ))}
        </select>
      </label>
      <label className="row">
時區
        <select value={s.timezone} onChange={(e) => save({ timezone: e.target.value })}>
          <option value="Asia/Shanghai">北京時間 (UTC+8)</option>
          <option value="Asia/Hong_Kong">香港時間 (UTC+8)</option>
          <option value="Asia/Taipei">台北時間 (UTC+8)</option>
          <option value="Asia/Singapore">新加坡時間 (UTC+8)</option>
          <option value="America/Los_Angeles">美西 (UTC-8)</option>
          <option value="America/New_York">美東 (UTC-5)</option>
          <option value="Europe/London">倫敦 (UTC+0)</option>
        </select>
      </label>
      <label className="row">
        <input
          type="checkbox"
          checked={s.daily_digest_enabled}
          onChange={(e) => save({ daily_digest_enabled: e.target.checked })}
        />
```

```
      啟用每日摘要推送
            </label>
            <label className="row">
              <input
                type="checkbox"
                checked={s.login_expiry_notify}
                onChange={(e) => save({ login_expiry_notify: e.target.checked })}
              />
      平台登錄態失效時提醒我
            </label>

            <h3>推送渠道（FR-1h，token 加密存儲）</h3>
            <p className="muted">
      已配置：{Object.entries(channels).length ? Object.entries(channels).map(([k, v]) => `${k}(${
{v})`).join("、") : "無"}
            </p>
            <div className="row wrap">
              <select
                value={chForm.channel}
                onChange={(e) => setChForm({ ...chForm, channel: e.target.value })}
              >
                <option value="pushplus">PushPlus（微信，需實名，每日 200 條免費）</option>
                <option value="wecom">企業微信群機器人 webhook</option>
                <option value="email">郵箱地址</option>
              </select>
              <input
                placeholder={
                  chForm.channel === "email" ? "你的郵箱" : chForm.channel === "wecom"
                    ? "https://qyapi.weixin.qq.com/cgi-bin/webhook/send?key=…" : "PushPlus token"
                }
                value={chForm.value}
                onChange={(e) => setChForm({ ...chForm, value: e.target.value })}
              />
              <button onClick={() => void saveChannel()}>保存渠道</button>
            </div>

            <p className="notice">{msg}</p>
          </div>
        </main>
      </div>
    );
}
import { useEffect, useState } from "react";
import { api } from "../lib/api";

type Task = {
  id: string;
  task_type: string;
  question: string;
  options: Record<string, { label: string; type: string }>;
  status: string;
```

```
  created_at: string;
  expires_at: string;
  application_status: string;
  company_name: string;
  title: string;
};

const TYPE_TEXT: Record<string, string> = {
  captcha: "驗證碼",
  login_required: "需要登錄",
  missing_field: "缺字段",
  missing_file: "缺文件",
  confirm_content: "確認內容",
  citics_manual: "中信官網投遞",
  manual_citics_login: "中信官網投遞",
  other: "其他",
};

export default function Tasks() {
  const [items, setItems] = useState<Task[]>([]);
  const [answers, setAnswers] = useState<Record<string, Record<string, string | boolean>>>({});
  const [msg, setMsg] = useState("");

  async function load() {
    try {
      setItems(await api<Task[]>("/api/manual-tasks?status=open"));
    } catch {
      setMsg("讀取任務失敗");
    }
  }
  useEffect(() => {
    void load();
  }, []);

  function setAnswer(taskId: string, key: string, value: string | boolean) {
    setAnswers((prev) => ({ ...prev, [taskId]: { ...(prev[taskId] ?? {}), [key]: value } }));
  }

  async function submit(task: Task) {
    setMsg("提交中（提交後會自動重新投遞）…");
    try {
      const r = await api<Record<string, unknown>>(`/api/manual-tasks/${task.id}/answer`, {
        method: "POST",
        body: JSON.stringify({ answer: answers[task.id] ?? {} }),
      });
      setMsg(`已提交並重新投遞：${JSON.stringify(r)}`);
      await load();
    } catch (e) {
      setMsg(`提交失敗：${(e as Error).message}`);
    }
```

```
}

async function fillPlan(task: Task) {
  try {
    const p = await api<{ copy_text: string }>(`/api/manual-tasks/${task.id}/fill-plan`);
    await navigator.clipboard.writeText(p.copy_text);
    setMsg("✅ 填寫方案已複製到剪貼板，粘貼到官網對應欄位即可");
  } catch (e) {
    setMsg(`獲取填寫方案失敗: ${(e as Error).message}`);
  }
}

async function cancel(task: Task) {
  if (!confirm("取消後該投遞將標記為放棄，確認? ")) return;
  await api(`/api/manual-tasks/${task.id}/cancel`, { method: "POST" });
  await load();
}

return (
  <div className="page">      <main className="narrow">
      <div className="card wide">
        <h2>待你處理</h2>
        <p className="muted">
   投遞遇到驗證碼、缺字段、缺文件時會暫停並在這裡等你。填寫/確認後系統自動在原位置繼續投遞。
        </p>
        {items.length === 0 && <p>暫無待處理任務🎉</p>}
        {items.map((t) => (
          <div key={t.id} className="task">
            <h3>
              [{TYPE_TEXT[t.task_type] ?? t.task_type}] {t.company_name} · {t.title}
            </h3>
            <p>{t.question}</p>
            {["citics_manual", "manual_citics_login"].includes(t.task_type) && (
              <>
                <p className="muted">
      🤖 推薦使用 RPA 填報助手（Chrome 擴展）：在官網登錄並打開申請頁後，點擴展「開始填報此頁」
      即可自動填寫全部欄位+上傳簡歷，你只需檢查並點提交。
      安裝：chrome://extensions → 開發者模式 → 載入未封裝項目 → 選擇項目 chrome-extension/ 目錄。
                </p>
                <button className="ghost" onClick={() => void fillPlan(t)}>
      📋 一鍵複製填寫方案（備用）
                </button>
              </>
            )}
            <div className="column">
              {Object.entries(t.options ?? {}).map(([key, opt]) => (
                <label key={key} className="row">
                  {opt.type === "checkbox" ? (
                    <input
                      type="checkbox"
```

```
                      checked={Boolean(answers[t.id]?.[key])}
                      onChange={(e) => setAnswer(t.id, key, e.target.checked)}
                    />
                  ) : opt.type === "textarea" ? (
                    <textarea
                      style={{ flex: 1, minHeight: 70 }}
                      placeholder={opt.label}
                      onChange={(e) => setAnswer(t.id, key, e.target.value)}
                    />
                  ) : (
                    <input
                      style={{ flex: 1 }}
                      placeholder={opt.label}
                      onChange={(e) => setAnswer(t.id, key, e.target.value)}
                    />
                  )}
                  <span className="muted">{opt.label}</span>
                </label>
              ))}
            </div>
            <div className="row">
              <button onClick={() => void submit(t)}>提交並繼續投遞</button>
              <button className="ghost" onClick={() => void cancel(t)}>
      放棄此投遞
              </button>
            </div>
            <p className="muted">
      過期時間：{t.expires_at}（48 小時未處理自動過期）
            </p>
          </div>
        ))}
        <p className="notice">{msg}</p>
      </div>
    </main>
  </div>
  );
}
/** JobsComing RPA 填報助手 — service worker（代持 API 請求，繞過 CORS）。 */
const API_BASE = "https://jobscoming-backend.onrender.com";

function withTimeout(ms) {
  return new AbortController();
}

async function timedFetch(url, opts, ms) {
  const ctrl = new AbortController();
  const t = setTimeout(() => ctrl.abort(), ms);
  try {
    return await fetch(url, { ...opts, signal: ctrl.signal });
  } finally {
```

```
    clearTimeout(t);
  }
}

async function apiFetch(path, options = {}) {
  const { token, refreshToken } = await chrome.storage.local.get(["token", "refreshToken"]);
  const doFetch = (tk) =>
    fetch(API_BASE + path, {
      ...options,
      headers: {
        "Content-Type": "application/json",
        ...(tk ? { Authorization: `Bearer ${tk}` } : {}),
        ...(options.headers || {}),
      },
    });

  // 45 秒超時 + 失敗重試一次（後台冷啟動可能需要時間）
  let res;
  try {
    res = await timedFetch(API_BASE + path, {
      ...options,
      headers: {
        "Content-Type": "application/json",
        ...(token ? { Authorization: `Bearer ${token}` } : {}),
        ...(options.headers || {}),
      },
    }, 45000);
  } catch {
    res = await timedFetch(API_BASE + path, {
      ...options,
      headers: {
        "Content-Type": "application/json",
        ...(token ? { Authorization: `Bearer ${token}` } : {}),
        ...(options.headers || {}),
      },
    }, 45000);
  }
  // 401 → 自動用 refresh token 續期並重試一次
  if (res.status === 401 && refreshToken) {
    try {
      const r = await fetch(API_BASE + "/api/auth/refresh", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ refresh_token: refreshToken }),
      });
      if (r.ok) {
        const data = await r.json();
        await chrome.storage.local.set({
          token: data.access_token,
          refreshToken: data.refresh_token || refreshToken,
```

```
        });
        res = await timedFetch(API_BASE + path, {
          ...options,
          headers: {
            "Content-Type": "application/json",
            ...(data.access_token ? { Authorization: `Bearer ${data.access_token}` } : {}),
            ...(options.headers || {}),
          },
        }, 45000);
      }
    } catch {
      /* 續期失敗走下方 AUTH 流程 */
    }
  }
  if (res.status === 401) {
    // 續期也失敗 → 清除憑證，通知 popup 顯示登錄框
    await chrome.storage.local.remove(["token", "refreshToken"]);
    const err = new Error("AUTH_EXPIRED");
    err.authExpired = true;
    throw err;
  }
  if (!res.ok) {
    const body = await res.text().catch(() => "");
    throw new Error(`HTTP ${res.status}: ${body.slice(0, 200)}`);
  }
  if (res.status === 204) return null;
  return res.json();
}

chrome.runtime.onMessage.addListener((msg, sender, sendResponse) => {
  (async () => {
    try {
      switch (msg.type) {
        case "SIGNIN": {
          const data = await apiFetch("/api/auth/signin", {
            method: "POST",
            body: JSON.stringify({ email: msg.email, password: msg.password }),
          });
          await chrome.storage.local.set({
            token: data.access_token,
            refreshToken: data.refresh_token || "",
          });
          sendResponse({ ok: true });
          break;
        }
        case "TASKS": {
          const tasks = await apiFetch("/api/manual-tasks?status=open");
          sendResponse({ ok: true, tasks });
          break;
        }
```

```
        case "FILL_PLAN": {
          const plan = await apiFetch(`/api/manual-tasks/${msg.taskId}/fill-plan`);
          sendResponse({ ok: true, plan });
          break;
        }
        case "COMPLETE": {
          const out = await apiFetch(`/api/manual-tasks/${msg.taskId}/answer`, {
            method: "POST",
            body: JSON.stringify({ answer: msg.answer }),
          });
          sendResponse({ ok: true, out });
          break;
        }
        case "DOWNLOAD": {
          // 經由後台下載簡歷（返回 dataURL，content script 塞進 file input）
          const { token } = await chrome.storage.local.get("token");
          const res = await fetch(msg.url);
          const blob = await res.blob();
          const dataUrl = await new Promise((resolve) => {
            const fr = new FileReader();
            fr.onload = () => resolve(fr.result);
            fr.readAsDataURL(blob);
          });
          sendResponse({ ok: true, dataUrl });
          break;
        }
        case "SCRAPE_SAVE": {
          const out = await apiFetch("/api/rpa/scrape", {
            method: "POST",
            body: JSON.stringify({ site: msg.site, page_type: msg.pageType, fields:
msg.fields }),
          });
          sendResponse({ ok: true, out });
          break;
        }
        case "RPA_FIELDS": {
          const fields = await apiFetch("/api/rpa/fields");
          sendResponse({ ok: true, fields });
          break;
        }
        case "PINYIN": {
          const out = await apiFetch(`/api/rpa/pinyin?name=${encodeURIComponent(msg.name || "")}
`);
          sendResponse({ ok: true, out });
          break;
        }
        case "RPA_VERIFY": {
          const out = await apiFetch("/api/rpa/verify", {
            method: "POST",
            body: JSON.stringify({
              site: msg.site, page_type: msg.pageType,
              compare_with: msg.compareWith, fields: msg.fields,
```

```
            }),
          });
          sendResponse({ ok: true, out });
          break;
        }
        default:
          sendResponse({ ok: false, error: "unknown message" });
      }
    } catch (e) {
      sendResponse({
        ok: false,
        error: e && e.authExpired ? "AUTH_EXPIRED" : String(e.message || e),
      });
    }
  })();
  return true; // 異步響應
});
/** JobsComing RPA 填報引擎 v0.6.1 — 單一 IIFE，全同步微操作 + 消息監聽。 */
(() => {
  const GUARD = "__jobsfly_rpa_v2__";
  if (window[GUARD]) return;
  window[GUARD] = true;

  /** 從網址自動識別中信簡歷類型（name=xiaozhao-cn 等） */
  function detectPageType() {
    try {
      const u = new URL(location.href);
      const name = u.searchParams.get("name") || "";
      const map = {
        "xiaozhao-cn": "校園招聘簡歷",
        "xiaozhao-fz": "分支校園招聘簡歷",
        "shezhao-cn": "社會招聘簡歷",
        "shezhao-fz": "分支社會招聘簡歷",
      };
      return map[name] || "";
    } catch {
      return "";
    }
  }

  let panel = null;

  function log(text, ok) {
    if (!panel) return;
    const line = document.createElement("div");
    line.textContent = text;
    line.style.color = ok === false ? "#dc2626" : ok ? "#15803d" : "#374151";
    panel.querySelector(".jrpa-log").appendChild(line);
  }
```

```
  function showPanel(title) {
    if (panel && panel.isConnected) return;
    panel = document.createElement("div");
    panel.style.cssText = [
      "position:fixed;right:16px;bottom:16px;z-index:99999;width:300px;",
      "background:#fff;border:1px solid #e5e7eb;border-radius:10px;",
      "box-shadow:0 8px 30px rgba(0,0,0,.18);padding:12px;font:13px/1.5 system-ui;",
    ].join("");
    panel.innerHTML =
      `<b>🤖 JobsComing RPA 填報</b><div style="color:#6b7280;margin:2px 0 6px">${title}</div>` +
      `<div class="jrpa-log" style="max-height:180px;overflow:auto"></div>`;
    document.body.appendChild(panel);
  }

  function collectFields() {
    const out = [];
    document.querySelectorAll("input:not([type=hidden]), textarea, select").forEach((el) => {
      out.push({
        el,
        label: labelFor(el),
        context: contextFor(el),
        type: (el.type || el.tagName.toLowerCase()),
        isFile: el.type === "file",
        isSelect: el.tagName.toLowerCase() === "select",
      });
    });
    return out;
  }

  /** 精確提取欄位標籤：label 元素 → 表單項 label → placeholder/name，避免抓到整塊長文本。 */
  function labelFor(el) {
    const ok = (t) => t && t.length >= 1 && t.length <= 40;
    if (el.labels && el.labels[0]) {
      const t = el.labels[0].innerText.trim();
      if (ok(t)) return t;
    }
    const item = el.closest(".el-form-item, .form-item, .ant-form-item, .ivu-form-item, .form-
group");
    if (item) {
      const lab = item.querySelector(":scope > label, .el-form-item__label, .form-label, .label,
.ant-form-item-label");
      if (lab) {
        const t = lab.innerText.trim();
        if (ok(t)) return t;
      }
    }
    if (el.placeholder && ok(el.placeholder.trim())) return el.placeholder.trim();
    if (el.name && ok(el.name)) return el.name;
    return "";
  }

  /** 上下文聯想：重複 label 時（如父親/母親的工作單位）取前面緊鄰的說明文本。 */
```

```
  function contextFor(el) {
    const item = el.closest(".el-form-item, .form-item, .ant-form-item, .ivu-form-item");
    if (!item || !item.parentElement) return "";
    const siblings = Array.from(item.parentElement.children);
    const idx = siblings.indexOf(item);
    for (let i = idx - 1; i >= 0 && i >= idx - 2; i--) {
      const t = (siblings[i].innerText || "").trim();
      if (t && t.length <= 20 && !t.includes(": ") && t !== item.innerText.trim()) {
        return t.slice(0, 20);
      }
    }
    return "";
  }

  function matchScore(label, field) {
    const l = label.toLowerCase();
    const f = (field.label || "").toLowerCase();
    const k = (field.key || "").toLowerCase();
    const short = (s) => s.length >= 1 && s.length <= 12;
    // 精確/包含匹配僅限短標籤（長文本 label 不得參與模糊包含，防止誤填）
    if (short(l) && short(f) && (l.includes(f) || f.includes(l))) return 3;
    if (short(l) && short(k) && (l.includes(k) || k.includes(l))) return 3;
    if (l === f) return 3;
    const map = {
      name: ["姓名", "名字", "name"],
      phone: ["手機", "电话", "电话", "mobile", "phone", "联系方式"],
      email: ["郵箱", "邮箱", "email", "电子邮箱", "電子郵箱"],
      visa: ["簽證", "签证", "visa"],
      salary: ["薪資", "薪资", "薪酬", "期望薪", "salary"],
      availability: ["到崗", "到岗", "入職", "入职", "available"],
      skills: ["技能", "特长", "擅長", "skill"],
      experience: ["工作經歷", "工作经历", "實習經歷", "实习经历", "工作經驗", "experience"],
      education: ["教育", "学历", "學歷", "學校", "学校", "毕业", "education"],
    };
    if (short(l)) {
      const kws = map[field.key] || [];
      if (kws.some((w) => l.includes(w))) return 1;
    }
    return 0;
  }

  /** 姓/名拆分與下拉值適配。*/
  function adaptValue(fieldInfo, field, value) {
    const l = (fieldInfo.label || "").toLowerCase();
    if (field.key === "name" && (l === "姓" || l === "姓氏")) return String(value || "").slice(0,
1);
    if (field.key === "name" && l === "名") return String(value || "").slice(1);
    return value;
  }

  function fillValue(fieldInfo, value) {
```

```
    const { el, isSelect } = fieldInfo;
    if (el.readOnly || el.disabled) return false;
    try {
      if (isSelect) {
        const opts = Array.from(el.options);
        const target = opts.find((o) => o.text.includes(value) || value.includes(o.text));
        if (target) {
          el.value = target.value;
          el.dispatchEvent(new Event("change", { bubbles: true }));
          return true;
        }
        return false;
      }
      const proto = el.tagName.toLowerCase() === "textarea"
        ? window.HTMLTextAreaElement.prototype
        : window.HTMLInputElement.prototype;
      const setter = Object.getOwnPropertyDescriptor(proto, "value").set;
      setter.call(el, value);
      el.dispatchEvent(new Event("input", { bubbles: true }));
      el.dispatchEvent(new Event("change", { bubbles: true }));
      el.style.outline = "2px solid #16a34a";
      el.style.outlineOffset = "2px";
      return true;
    } catch {
      return false;
    }
  }

  // ── 同步 API（popup 以 executeScript 直接調用，頁面側零異步）──

  function syncCollect() {
    return collectFields()
      .filter((f) => !f.isFile)
      .map((f) => ({
        // 上下文聯想：重複 label 加前綴（如「父親｜工作單位」）
        label: f.context ? `${f.context}｜${f.label}` : f.label,
        value: f.el.value || "",
      }));
  }

  function syncFill(entries) {
    const fields = collectFields();
    let filled = 0;
    const skipped = [];
    for (const f of fields) {
      if (f.isFile) continue;
      if (!f.label) continue;
      // 拼音欄位：僅當有對應拼音數據時填充
      if (f.label.includes("拼音")) {
        const py = (entries || []).find((e) => String(e.label || "").split("｜").pop() ===
f.label);
```

```
      if (py && py.value) {
        if (fillValue(f, String(py.value))) filled++;
      } else {
        skipped.push(f.label);
      }
      continue;
    }
    const best = (entries || [])
      .map((m) => {
        const parts = String(m.label || "").split("｜");
        const ctx = parts.length > 1 ? parts[0] : "";
        const plainLabel = parts.pop();
        let s = matchScore(f.label, { ...m, label: plainLabel });
        // 上下文聯想優先：父親/母親等前綴一致 +5
        if (ctx && f.context && (f.context.includes(ctx) || ctx.includes(f.context))) s += 5;
        else if (ctx && f.label.includes("父") && ctx.includes("父")) s += 5;
        else if (ctx && f.label.includes("母") && ctx.includes("母")) s += 5;
        else if (ctx && f.label.includes(ctx)) s += 5;
        return { m, plainLabel, s };
      })
      .filter((x) => x.s > 0 && x.m.value)
      .sort((a, b) => b.s - a.s)[0];
    if (best) {
      const v = adaptValue(f, best.m, String(best.m.value));
      if (fillValue(f, v)) filled++;
    } else {
      skipped.push(f.label);
    }
  }
  const detail = fields
    .filter((f) => f.label && !f.isFile)
    .slice(0, 40)
    .map((f) => ({
      label: f.label,
      filled: f.el.style.outline ? true : false,
    }));
  return { filled, skipped, detail };
}

function syncUploadResume(fileInfo) {
  const inputs = collectFields().filter((f) => f.isFile).map((f) => f.el);
  if (!inputs.length) return { uploaded: false, note: "本頁無文件上傳欄位" };
  if (!fileInfo || !fileInfo.dataUrl) return { uploaded: false, note: "無簡歷文件" };
  try {
    const b64 = fileInfo.dataUrl.split(",")[1];
    const bin = atob(b64);
    const arr = new Uint8Array(bin.length);
    for (let i = 0; i < bin.length; i++) arr[i] = bin.charCodeAt(i);
    const file = new File([arr], fileInfo.name || "resume.pdf", { type: "application/pdf" });
    const dt = new DataTransfer();
```

```
      dt.items.add(file);
      inputs[0].files = dt.files;
      inputs[0].dispatchEvent(new Event("change", { bubbles: true }));
      inputs[0].style.outline = "2px solid #16a34a";
      return { uploaded: true, name: file.name };
    } catch (e) {
      return { uploaded: false, note: String(e) };
    }
  }

  function syncPanel(title, lines) {
    showPanel(title);
    for (const l of lines || []) log(l.text, l.ok);
  }

  window.__jrpa_sync__ = {
    collect: syncCollect,
    fill: syncFill,
    upload: syncUploadResume,
    panel: syncPanel,
    detectPageType,
  };
  window.__jrpa_run_v3__ = true;
})();
const $ = (id) => document.getElementById(id);

/* ── E2EE 密鑰（隱私第二層）：PBKDF2(密碼, 郵箱) → AES-GCM ── */
const _b64 = (buf) => btoa(String.fromCharCode(...new Uint8Array(buf)));
const _unb64 = (s) => Uint8Array.from(atob(s), (c) => c.charCodeAt(0));

async function deriveKeyB64(password, salt) {
  const enc = new TextEncoder();
  const base = await crypto.subtle.importKey("raw", enc.encode(password), "PBKDF2", false,
["deriveKey"]);
  const key = await crypto.subtle.deriveKey(
    { name: "PBKDF2", salt: enc.encode(salt), iterations: 100000, hash: "SHA-256" },
    base, { name: "AES-GCM", length: 256 }, false, ["encrypt", "decrypt"],
  );
  const raw = await crypto.subtle.exportKey("raw", key);
  return _b64(raw);
}

async function encryptText(keyB64, text) {
  const key = await crypto.subtle.importKey("raw", _unb64(keyB64), "AES-GCM", false,
["encrypt"]);
  const iv = crypto.getRandomValues(new Uint8Array(12));
  const ct = await crypto.subtle.encrypt({ name: "AES-GCM", iv }, key, new
TextEncoder().encode(text));
  return _b64(iv) + "|" + _b64(ct);
}

async function decryptText(keyB64, data) {
  if (!data || !data.includes("|")) return null;
```

```
  try {
    const [ivS, ctS] = data.split("|");
    const key = await crypto.subtle.importKey("raw", _unb64(keyB64), "AES-GCM", false,
["decrypt"]);
    const pt = await crypto.subtle.decrypt({ name: "AES-GCM", iv: _unb64(ivS) }, key,
_unb64(ctS));
    return new TextDecoder().decode(pt);
  } catch {
    return null;
  }
}

const SENSITIVE_RE = /證件|证件|身份證|身份证|護照|护照|家庭|親屬|亲属|緊急聯繫|紧急联系|證件號|证件号|地址|
邮编|郵編|電話|电话|手機|手机|姓名拼音|父|母/;

async function getKeyB64() {
  const st = await chrome.storage.session.get("rawKeyB64");
  return st.rawKeyB64 || null;
}

const SITE_RE = /citics\.com|gtht\.com|greenhouse\.io|lever\.co/;

function showLogin() {
  $("loginBox").style.display = "block";
  $("taskBox").style.display = "none";
  $("status").textContent = "登錄已失效，請重新登錄";
  $("status").className = "err";
}

function isAuthError(e) {
  return !!e && (e.authExpired || String(e.message || e).includes("AUTH_EXPIRED"));
}

chrome.storage.local.get("token").then(({ token }) => {
  if (token) showTasks().catch((e) => isAuthError(e) && showLogin());
  else showLogin();
});

$("signin").addEventListener("click", async () => {
  $("status").textContent = "登錄中…";
  const email = $("email").value.trim();
  const password = $("password").value;
  const r = await chrome.runtime.sendMessage({
    type: "SIGNIN",
    email,
    password,
  });
  if (r && r.ok) {
    // 派生 E2EE 密鑰（僅存瀏覽器 session，關閉瀏覽器即清除）
    const keyB64 = await deriveKeyB64(password, "jobfly:" + email);
    await chrome.storage.session.set({ rawKeyB64: keyB64 });
    $("password").value = "";
    showTasks();
```

```
  } else {
    $("status").textContent = "登錄失敗: " + (r?.error || "未知錯誤");
    $("status").className = "err";
  }
});

$("refresh").addEventListener("click", () => {
  loadTasks().catch((e) => isAuthError(e) && showLogin());
});

async function showTasks() {
  $("loginBox").style.display = "none";
  $("taskBox").style.display = "block";
  await loadTasks();
}

async function loadTasks() {
  $("status").textContent = "讀取任務…";
  let r;
  try {
    r = await chrome.runtime.sendMessage({ type: "TASKS" });
  } catch (e) {
    $("status").textContent = "讀取失敗: " + (e?.message || e);
    $("status").className = "err";
    return;
  }
  if (!r || !r.ok) {
    if (isAuthError(r?.error)) {
      showLogin();
      return;
    }
    $("status").textContent = "讀取失敗: " + (r?.error || "");
    $("status").className = "err";
    return;
  }
  const box = $("tasks");
  box.innerHTML = "";
  if (!r.tasks.length) {
    box.innerHTML = '<p class="muted">暫無待處理投遞任務</p>';
    $("status").textContent = "";
    return;
  }
  for (const t of r.tasks) {
    const div = document.createElement("div");
    div.className = "task";
    div.innerHTML =
      `<b>${t.company_name} · ${t.title}</b>` +
      `<span class="muted">${t.task_type}</span>`;
    const btn = document.createElement("button");
    btn.className = "primary";
```

```
    btn.textContent = "開始填報此頁";
    btn.addEventListener("click", () => runAction("FILL", { taskId: t.id }));
    div.appendChild(btn);
    box.appendChild(div);
  }
  $("status").textContent = "";
}

/** 選目標頁籤：優先當前活動頁（若是官網）；否則找最近訪問的官網頁籤。 */
async function pickTab() {
  const [active] = await chrome.tabs.query({ active: true, currentWindow: true });
  if (active && active.url && SITE_RE.test(active.url)) return active;
  const all = await chrome.tabs.query({ currentWindow: true });
  const site = all.filter((t) => t.url && SITE_RE.test(t.url));
  return site[site.length - 1] || active;
}

/** 頁面注入：全幀執行同步小函數並返回各幀結果數組（覆蓋 iframe 表單）。 */
async function inject(tab, func, args = []) {
  // 全幀注入庫（表單可能在 iframe 中）
  try {
    const probe = await chrome.scripting.executeScript({
      target: { tabId: tab.id, allFrames: true },
      func: () => !!(window.__jrpa_sync__ && window.__jrpa_run_v3__),
    });
    const missing = probe.some((r) => !(r && r.result));
    if (missing) {
      await chrome.scripting.executeScript({
        target: { tabId: tab.id, allFrames: true },
        files: ["content.js"],
      });
      await new Promise((r) => setTimeout(r, 400));
    }
  } catch {
    /* 部分幀不可注入（跨域），退回頂層幀 */
  }
  try {
    const out = await chrome.scripting.executeScript({
      target: { tabId: tab.id, allFrames: true },
      func,
      args,
    });
    return (out || []).map((o) => o.result).filter((r) => r != null);
  } catch {
    const out = await chrome.scripting.executeScript({ target: { tabId: tab.id }, func, args });
    return [(out || [])[0] ? out[0].result : null].filter((r) => r != null);
  }
}

/** 頁面右下角面板（同步注入，立刻可見）。 */
```

```
function pagePanel(lines) {
  return (lines) => window.__jrpa_sync__.panel("JobsComing RPA", lines);
}

async function bg(type, payload) {
  return chrome.runtime.sendMessage({ type, ...payload });
}

/** 主操作編排：所有後台請求在 popup 完成，頁面只做同步小動作。 */
async function runAction(type, extra) {
  $("status").textContent = "執行中…";
  $("status").className = "";
  try {
    const tab = await pickTab();
    if (!tab || !tab.url || !/^https?:/.test(tab.url)) {
      throw new Error("請先切換到官網頁面（當前是瀏覽器內部頁面）");
    }
    if (type === "SCRAPE") {
      const pageType = $("pageType").value;
      const frames = await inject(
        tab,
        () => window.__jrpa_sync__.collect(),
      );
      const fields = (frames || []).flat();
      const panelArgs = [[{ text: `頁面共識別到 ${fields ? fields.length : 0} 個欄位`, ok: null },
          { text: "正在連線 JobsComing 後台…", ok: null }]];
      await chrome.scripting.executeScript({
        target: { tabId: tab.id },
        func: (ls) => {
          window.__jrpa_sync__.panel("回傳簡歷資料", ls);
          return true;
        },
        args: panelArgs,
      });
      const r = await bg("SCRAPE_SAVE", { site: tab.url ? new URL(tab.url).hostname : "",
pageType, fields: fields || [] });
      if (r && r.ok) {
        $("status").textContent = `✅ 已回傳 ${r.out.saved} 個字段到元信息表`;
        $("status").className = "ok";
        await chrome.scripting.executeScript({
          target: { tabId: tab.id },
          func: (n) => {
            window.__jrpa_sync__.panel("回傳簡歷資料", [{ text: `✅ 已回傳 ${n} 個字段到元信息表`, ok:
true }]);
            return true;
          },
          args: [r.out.saved],
        });
      } else {
        $("status").textContent = "回傳失敗: " + (r?.error || "");
        $("status").className = "err";
      }
```

```
      return;
    }
    if (type === "FILL_RESUME") {
      const pageType = $("pageType").value;
      const r = await bg("RPA_FIELDS", {});
      if (!r || !r.ok) throw new Error("讀取元信息失敗: " + (r?.error || ""));
      // 敏感字段本地解密
      const keyB64 = await getKeyB64();
      let entries = r.fields || [];
      // 拼音聯想：從姓名生成姓/名拼音
      const nameEntry = entries.find((e) => String(e.label || "").split("｜").pop() === "姓名");
      const nameVal = nameEntry ? String(nameEntry.value || "").trim() : "";
      if (nameVal) {
        try {
          const py = await bg("PINYIN", { name: nameVal });
          if (py && py.ok && (py.out.surname_pinyin || py.out.given_pinyin)) {
            entries = entries.concat([
              { label: "姓（拼音）", value: py.out.surname_pinyin, source: "pinyin" },
              { label: "名（拼音）", value: py.out.given_pinyin, source: "pinyin" },
            ]);
          }
        } catch { /* 忽略 */ }
      }
      if (keyB64) {
        entries = await Promise.all(entries.map(async (e) => {
          if (e.encrypted) {
            const pt = await decryptText(keyB64, e.encrypted);
            if (pt != null) return { ...e, value: pt };
          }
          return e;
        }));
      }
      const frames = await inject(
        tab,
        (ents) => window.__jrpa_sync__.fill(ents),
        [entries],
      );
      let filled = 0;
      const skipped = [];
      for (const res of frames || []) {
        filled += res ? res.filled : 0;
        if (res) skipped.push(...(res.skipped || []));
      }
      $("status").textContent =
        `✅ 已填寫 ${filled} 個欄位` +
        (skipped.length ? `｜未匹配 ${skipped.length} 個: ${skipped.slice(0, 8).join("、")}…` :
"");
      $("status").className = "ok";
      return;
    }
    if (type === "VERIFY") {
```

```
      const pageType = $("pageType").value;
      const compareWith = $("compareWith").value;
      const frames = await inject(tab, () => window.__jrpa_sync__.collect());
      const fields = (frames || []).flat();
      const r = await bg("RPA_VERIFY", {
        site: tab.url ? new URL(tab.url).hostname : "",
        pageType,
        compareWith,
        fields: fields || [],
      });
      if (r && r.ok && r.out.ok) {
        $("status").textContent = r.out.summary;
        $("status").className = "ok";
      } else {
        $("status").textContent = "校驗失敗: " + ((r && r.out && r.out.error) || r?.error || "");
        $("status").className = "err";
      }
      return;
    }
    if (type === "FILL") {
      const r = await bg("FILL_PLAN", { taskId: extra.taskId });
      if (!r || !r.ok) throw new Error("獲取填寫方案失敗: " + (r?.error || ""));
      const frames = await inject(
        tab,
        (machine) => window.__jrpa_sync__.fill(machine),
        [r.plan.machine_fields || []],
      );
      let filled = 0;
      const skipped = [];
      for (const res of frames || []) {
        filled += res ? res.filled : 0;
        if (res) skipped.push(...(res.skipped || []));
      }
      let note = `✅ 已填寫 ${filled} 個欄位`;
      if (skipped.length) note += ", 未匹配: " + skipped.slice(0, 6).join("、");
      $("status").textContent = note;
      $("status").className = "ok";
      return;
    }
    $("status").textContent = "未知操作";
    $("status").className = "err";
  } catch (e) {
    $("status").textContent = "操作失敗: " + (e?.message || e);
    $("status").className = "err";
  }
}

$("scrape").addEventListener("click", () => runAction("SCRAPE", {}));
$("fillResume").addEventListener("click", () => runAction("FILL_RESUME", {}));
$("verify").addEventListener("click", () => runAction("VERIFY", {}));
```